TEACHER'S GUIDE

Working for over 25 YEARS WITH Cambridge Assessment International Education

Cambridge Assessment International Education

Endorsed for teacher support

# Cambridge IGCSE™ and O Level

# Computer Science

David Watson
Helen Williams

## with Boost Subscription

To get full access to all the Teacher's Guide resources, this handbook must be used alongside your online Boost subscription.

**To activate your Boost subscription, please see inside for step-by-step instructions.**

Boost

HODDER EDUCATION

To access your Boost Subscription, please log in at **www.hoddereducation.co.uk/codeaccess** and enter your unique code

Once registered, you can log in and access the online Teacher's Guide resources at **boost-learning.com**

Cambridge IGCSE™ and O Level

# Computer Science

## Teacher's Guide with Boost Subscription

David Watson
Helen Williams

Boost

HODDER EDUCATION
AN HACHETTE UK COMPANY

The information in this publication is taken from the Cambridge IGCSE and O Level Computer Science (0478/0984/2210) syllabuses for examination from 2023. You should always refer to the appropriate syllabus document for the year of your students' examination to confirm the details and for more information. The syllabus document is available on the Cambridge International website at www.cambridgeinternational.org.

Cambridge International copyright material in this publication is reproduced under licence and remains the intellectual property of Cambridge Assessment International Education.

Third-party websites and resources referred to in this publication have not been endorsed by Cambridge Assessment International Education.

Every effort has been made to trace all copyright holders, but if any have been inadvertently overlooked, the Publishers will be pleased to make the necessary arrangements at the first opportunity.

Although every effort has been made to ensure that website addresses are correct at time of going to press, Hodder Education cannot be held responsible for the content of any website mentioned in this book. It is sometimes possible to find a relocated web page by typing in the address of the home page for a website in the URL window of your browser.

Hachette UK's policy is to use papers that are natural, renewable and recyclable products and made from wood grown in well-managed forests and other controlled sources. The logging and manufacturing processes are expected to conform to the environmental regulations of the country of origin.

Orders: please contact Hachette UK Distribution, Hely Hutchinson Centre, Milton Road, Didcot, Oxfordshire, OX11 7HH. Telephone: +44 (0)1235 827827. Email: education@hachette.co.uk. Lines are open from 9 a.m. to 5 p.m., Monday to Friday. You can also order through our website: www.hoddereducation.com.

ISBN: 978 1 3983 1850 2

© David Watson and Helen Williams 2021

First published in 2021 by
Hodder Education
An Hachette UK Company
Carmelite House
50 Victoria Embankment
London EC4Y 0DZ

www.hoddereducation.com

Impression number 10 9 8 7 6 5 4 3 2 1

Year 2025 2024 2023 2022 2021

All rights reserved. Apart from any use permitted under UK copyright law, no part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording, or held within any information storage and retrieval system, without permission in writing from the publisher or under licence from the Copyright Licensing Agency Limited. Further details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Limited, www.cla.co.uk

Typeset by Aptara Inc.

Printed in the UK

A catalogue record for this title is available from the British Library.

FSC
www.fsc.org
MIX
Paper from responsible sources
FSC™ C104740

# Contents


| | |
|---|---|
| Introduction | 4 |
| How to teach this course | 7 |
| Suggested schemes of work | 9 |
| Sample lesson plans | 21 |
| Scenario-based questions | 25 |
| ESL guidance | 27 |
| Glossary | 34 |

# Introduction

This Teacher's Guide offers complete support to teachers by providing the following:

- teaching tips for delivering the Cambridge IGCSE™, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) for examinations from 2023
- two schemes of work, to help with planning the course
- a lesson plan template followed by two sample lesson plans, so teachers can design their own lessons
- tips and advice for teaching learners whose first language is not English
- a glossary of terms used in the Student's Book, to help learners develop computer science vocabulary.

The online elements of the Teacher's Guide offer further support by providing:

- interactive versions of the schemes of work, so teachers can adapt them to suit the needs of their students
- editable template for lesson plans, easy for teachers to make their own
- language development activities, to help students to develop their reading and listening skills
- flashcards and a bilingual glossary, to develop students' computer science vocabulary
- sample Python, Visual Basic and Java program files
- PowerPoint presentations to explain key technical concepts
- answers to the activities and exam-style questions in the Student's Book
- answers to questions in the *Computer Systems Workbook*
- answers to questions in the *Algorithms, Programming and Logic* Workbook.

To access the online resources that accompany this Teacher's Guide, please visit **boost-learning.com**.

## Syllabus and assessment

The information in the below sections is taken from the Cambridge IGCSE and O Level Computer Science (0478/0984/2210) syllabuses for examination from 2023. You should always refer to the appropriate syllabus document for the year of your students' examinations to confirm the details and for more information. The syllabus document is available on the Cambridge International website at www.cambridgeinternational.org.

The aims of the Cambridge IGCSE, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) are to enable students to develop:

- computational thinking skills
- an understanding of the main principles of solving problems using computers
- the skills necessary to solve computer-based problems using a high-level programming language
- an understanding of the component parts of computer systems and how they interrelate
- an understanding of the internet as a means of communication and its associated risks
- an understanding of the development and use of automated and emerging technologies.

The assessment objectives (AOs) are:

### AO1 Knowledge and understanding:

- Demonstrate knowledge and understanding of the principles and concepts of computer science.

### AO2 Application:

- Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems.

### AO3 Evaluation:

- Provide solutions to problems by:
  - evaluating computer systems
  - making reasoned judgements
  - presenting conclusions.

## Scheme of assessment for IGCSE and O Level Computer Science

All candidates take two components.

- Computer Systems – 1 hour 45 minutes
  - Short-answer and structured questions based on Topics 1–6 of the syllabus
  - 75 marks – 50% of the final assessment
  - All questions are compulsory
  - No calculators are permitted
  - Externally assessed
- Algorithms, Programming and Logic – 1 hour 45 minutes
  - Short-answer and structured questions and a scenario-based question
  - 75 marks – 50% of the final assessment
  - Questions based on Topics 7–10 of the syllabus
  - All questions are compulsory
  - No calculators are permitted
  - Externally assessed

# Cambridge learner attributes

*Cambridge International copyright material in this publication is reproduced under licence and remains the intellectual property of Cambridge Assessment International Education.*

Cambridge Assessment International Education have developed 'Cambridge learner attributes', which summarise the attitudes and life skills learners need to develop alongside their academic skills. These attributes will help learners to be successful both while they are studying and beyond. The Cambridge Learner Attributes can be found on the Cambridge International website. We have expanded on this below, in the text shown in italics, in relation to Computer Science specifically.

Cambridge programmes are designed to encourage learners to be:

**Confident in working with information and ideas – their own and those of others**

Cambridge learners are confident, secure in their knowledge, unwilling to take things for granted and ready to take intellectual risks. They are keen to explore and evaluate ideas and arguments in a structured, critical and analytical way. They are able to communicate and defend views and opinions as well as respect those of others. *Cambridge Computer Science students are interested in learning about computer science and using technical language to communicate their knowledge and understanding.*

**Responsible for themselves, responsive to and respectful of others**

Cambridge learners take ownership of their learning, set targets and insist on intellectual integrity. They are collaborative and supportive. They understand that their actions have impacts on others and on the environment. They appreciate the importance of culture, context and community. *Cambridge Computer Science students work systematically, safely and securely when using technology.*

**Reflective as learners, developing their ability to learn**

Cambridge learners understand themselves as learners. They are concerned with the processes as well as the products of their learning and develop the awareness and strategies to be lifelong learners. *Cambridge Computer Science students learn from their experiences when creating programs and using technology and understand the impacts of technology on society.*

**Innovative and equipped for new and future challenges**

Cambridge learners welcome new challenges and meet them resourcefully, creatively and imaginatively. They are capable of applying their knowledge and understanding to solve new and unfamiliar problems. They can adapt flexibly to new situations requiring new ways of thinking. *Cambridge Computer Science students are able to find creative solutions to problems and to design computer programs independently.*

**Engaged intellectually and socially, ready to make a difference**

Cambridge learners are alive with curiosity, embody a spirit of enquiry and want to dig more deeply. They are keen to learn new skills and are receptive to new ideas. They work well independently, but also with others. They are equipped to participate constructively in society and the economy – locally, nationally and globally. *Cambridge Computer Science students are keen to develop computer science skills and further their understanding of developments in the use of technology*.

Cambridge have also produced Cambridge Teacher Standards, which can be found here: www.cambridgeinternational.org/Images/466465-cambridge-teacher-standards.pdf.

# How to teach this course

## Aims

This Teacher's Guide provides a range of teaching and reference materials which have been developed to aid the teaching of the Cambridge IGCSE, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) for examination from 2023. It also supports the following resources which have been endorsed by Cambridge Assessment International Education:

- *Cambridge IGCSE™ and O Level Computer Science Second Edition* ISBN 9781398318281
- *Cambridge IGCSE™ and O Level Computer Science Computer Systems Workbook* ISBN 9781398318496
- *Cambridge IGCSE™ and O Level Computer Science Algorithms, Programming and Logic Workbook* ISBN 9781398318472

## Planning and schemes of work

Two schemes of work have been devised to provide a logical route through the *Cambridge IGCSE and O Level Computer Science Second Edition* Student's Book. The aim is to complete the study of the syllabus by the end of the second term in the second year of study. This will then allow adequate time for revision and examination practice.

It is assumed that students will have about three hours per week directed study time for this subject. This will give students a total study time of 130 hours or more, comprising at least 65 hours for Section 1 Computer Systems and at least 65 hours for Section 2 Algorithms, Programming and Logic.

**Scheme of Work 1** assumes that the topics relating to both papers of the Cambridge syllabus will be taught side by side. This scheme of work therefore assumes one and a half hours per week on Computer Systems and one and a half hours per week on Algorithms, Programming and Logic. Naturally, the scheme of work can be modified depending on resource allocation and other factors.

**Scheme of Work 2** assumes that the topics relating to both topics Computer Systems and Algorithms, Programming and Logic of the Cambridge syllabus will be taught one after the other. This scheme of work therefore assumes three hours per week. Naturally, the scheme of work can be modified depending on resource allocation and other factors.

Details of the syllabus references are given, along with cross-references to the appropriate pages in the Student's Book and the relevant workbook (*Computer Systems Workbook* or *Algorithms, Programming and Logic Workbook*). Additional lesson notes are also provided.

### Lesson plan template

A template has been designed that teachers can use to prepare lesson plans. It is easy to use, followed by two sample lesson plans for Digital Currency and Programming concepts – iteration.

### Answers

Answers to the end-of-chapter questions and (where relevant) the activities in the Student's Book are available on **boost-learning.com**. Answers are also provided to the two workbooks (the *Computer Systems Workbook* and the *Algorithms, Programming and Logic Workbook*).

### Program files

The Python, Visual Basic and Java program files available on **boost-learning.com** are sample answer files for the activities and end-of-chapter questions in Chapters 7 and 8. These programs use the recommended languages for IGCSE and O Level Computer Science. As well as using these program files as sample answers for the exercises in the Student's Book, teachers could use them as a basis to develop their own programming exercises for students.

### Additional content in the Student's Book

Most chapters include an Extension feature, covering content that is not in the current syllabus, but which provides a useful bridge for those students who wish to continue their computer science studies to A Level.

## Advice to teachers

The Cambridge IGCSE, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) are very comprehensive, but with careful planning, using all necessary print and online resources, you and your students should find the course very manageable and interesting.

To best support the students' study, the Hodder Education series of Computer Science resources follows the syllabuses very closely. It is therefore possible to plan lessons in the following ways:

- Match each section of a syllabus topic to the relevant sections of your preferred scheme of work (either Scheme of Work 1 (on pages 8–13 of this Teacher's Guide) or Scheme of Work 2 (on pages 14–19) and resources found here.
- Match each section of a syllabus topic to the Student's Book. As your students complete each section, set them classwork and homework questions from the corresponding workbook (the *Computer Systems Workbook* and the *Algorithms, Programming and Logic Workbook*). These questions were developed to be used in conjunction with the Student's Book and therefore allow you to monitor students' understanding easily.
- The questions in both the workbooks provide extra practice for students. The answers to these questions can be found in the online part of this Teacher's Guide at **boost-learning.com**.
- At the end of each Student's Book chapter (which corresponds to the end of a main syllabus topic), you can find additional exam-style questions, with answers available in this teaching resource.

However, it is up to you how to integrate the Student's Book, workbooks and this Teacher's Guide to get the best out of your students. The notes above are simply for guidance.

Because computer science is a rapidly changing subject, it is important to keep abreast of the latest technologies and use these to update lessons wherever possible.

You can use other resources, such as the internet, as part of your teaching in lessons (but be careful to check the accuracy and validity of data found on websites).

## Additional resources

The Cambridge IGCSE and O Level Computer Science Study and Revision guide Second edition (ISBN: 9781398318489) contains exam-style questions with answers. This has not been through the Cambridge International endorsement process.

## Learners whose first language is not English

This Teacher's Guide includes teaching tips and advice for teaching Cambridge IGCSE™ and O Level computer science to learners whose first language is not English (see pages 26–31). Online you will find plenty of ideas for helping learners develop computer science vocabulary in English as well as reading and listening about computer science in English. These ideas include strategies and techniques for teachers and students to help boost learner confidence in using English associated with computer science. Additionally, there are ten exemplar language development activities. You can use these as flexible models and build your own activities to fit in with the specific content you are teaching. Most of these activities are designed to familiarise students with techniques for developing computer science vocabulary, taking notes, organising ideas and comprehending problems. You can either use these activities in class, for example, during self-study sessions, or assign them as homework.

# Suggested Scheme of Work 1

The following outline Scheme of Work is a potential scheme – you should adapt it to the needs of your students.

## Teaching the content of Computer Systems and Algorithms, Programming and Logic in parallel

In the scheme of work, Workbook 1 refers to the Computer Systems Workbook, and Workbook 2 refers to the Algorithms, Programming and Logic Workbook.

The times given are approximate time allocations for each topic and sub-topic of the syllabus.

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 1** | **Data representation** | **12 hours** | | | |
| 1.1 Number systems | **1** How and why computers use binary to represent all forms of data<br>**2** Denary, binary and hexadecimal number systems and conversion of positive integers between these number systems<br>**3** How and why hexadecimal is used as a beneficial method of data representation<br>**4** Addition of two positive 8-bit binary integers and the concept of overflow and why it occurs in binary addition<br>**5** Logical binary shifts on positive 8-bit binary integers and the effect this has on the positive binary integer<br>**6** Two's complement to represent positive and negative 8-bit binary integers | 7 hours | **1** Chapter 1 pages 2–3<br>**2** Chapter 1 pages 3–12<br>**3** Chapter 1 pages 12–15<br>**4** Chapter 1 pages 15–17<br>**5** Chapter 1 pages 17–20<br>**6** Chapter 1 pages 20–25 | **2** Workbook 1, Ch 1, questions 1–6, 8–9, 12a, 13a, 23<br>**3** Workbook 1, Ch 1, question 7<br>**4** Workbook 1, Ch 1, questions 10–11, 13b, 19<br>**5** Workbook 1, Ch 1, questions 12b,c–13c,d, 19<br>**6** Workbook 1, Ch 1, questions 14, 19 | It is essential to provide practice in conversion, addition, shifting and use of two's complement. There are plenty of online sources that provide extra practice. |
| 1.2 Text, sound and images | **1** How and why a computer represents text and the use of character sets, including ASCII and Unicode<br>**2** How and why a computer represents sound, including the effects of the sample rate and sample resolution<br>**3** How and why a computer represents an image, including the effects of the resolution and colour depth | 3 hours | **1** Chapter 1 pages 25–28<br>**2** Chapter 1 pages 29–30<br>**3** Chapter 1 pages 30–31 | **1** Workbook 1, Ch 1, questions 18, 19<br>**2** Workbook 1, Ch 1, questions 19, 20b,c, 22<br>**3** Workbook 1, Ch 1, questions 17, 19, 20d | |
| 1.3 Data storage and file compression | **1** How data storage is measured<br>**2** Calculation of file sizes of image and sound files<br>**3** The purpose of and need for data compression<br>**4** Lossy and lossless file compression methods | 2 hours | **1** Chapter 1 page 32<br>**2** Chapter 1 pages 33–34<br>**3** Chapter 1 page 34<br>**4** Chapter 1 pages 34–37 | **1** Workbook 1, Ch 1, question 3<br>**2** Workbook 1, Ch 1, question 15<br>**3** Workbook 1, Ch 1, question 16a<br>**4** Workbook 1, Ch 1, questions 16b,c, 20e, 21 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 7** | **Algorithm design and problem solving** | **21 hours** | | | |
| | **1** The program development life cycle: analysis, design, coding and testing<br>**2** Computer systems and sub-systems; problem decomposition; methods to design and construct a solution to a problem<br>**3** The purpose of a given algorithm<br>**4** Standard methods of solution<br>**5** The need for validation and verification checks on input data; different types of validation and verification checks<br>**6** Use of test data<br>**7** Use of trace tables to document dry-runs of algorithms<br>**8** Identification and correction of errors in algorithms<br>**9** Writing and amending algorithms for given problems/scenarios, using pseudocode, program code and flowcharts | 21 hours | **1** Chapter 7 pages 258–259<br>**2** Chapter 7 pages 260–270<br>**3** Chapter 7 page 271<br>**4** Chapter 7 pages 272–276<br>**5** Chapter 7 pages 276–281<br>**6** Chapter 7 pages 281–282<br>**7** Chapter 7 pages 282–285<br>**8** Chapter 7 pages 285–288<br>**9** Chapter 7 pages 288–292 | **1** Workbook 2 Ch 7 question 1<br>**2** Workbook 2 Ch 7 questions 2–8<br>**3** Workbook 2 Ch 7 questions 9, 10<br>**4** Workbook 2 Ch 7 questions 11–12<br>**5** Workbook 2 Ch 7 question 13<br>**6** Workbook 2 Ch 7 question 14<br>**7** Workbook 2 Ch 7 question 15<br>**8** Workbook 2 Ch 7 question 16<br>**9** Workbook 2 Ch 7 question 15d | The flowchart symbols to use are on page 21 of the syllabus.<br>The pseudocode used is set out on pages 33–46 of the syllabus. |
| **Topic 2** | **Data transmission** | **8 hours** | | | |
| 2.1 Types and methods of data transmission | **1** Breaking data into packets to be transmitted; the structure of a packet; the process of packet switching<br>**2** How data is transmitted from one device to another using different methods of data transmission; the suitability of each method of data transmission for a given scenario<br>**3** The universal serial bus (USB) interface and how it is used to transmit data | 4 hours | **1** Chapter 2 pages 45–49<br>**2** Chapter 2 pages 49–52<br>**3** Chapter 2 pages 52–53 | **1** Workbook 1 Ch 2 questions 1–2, 9a, 10<br>**2** Workbook 1 Ch 2 questions 3, 9b, 10<br>**3** Workbook 1 Ch 2 questions 4, 9c | There are useful online demonstrations available. |
| 2.2 Methods of error detection | **1** The need to check for errors after data transmission and how these errors can occur<br>**2** The processes involved in error detection; methods for detecting errors in data after transmission; parity check (odd and even), checksum, echo check<br>**3** How a check digit is used to detect errors in data entry, including ISBNs and bar codes<br>**4** How an automatic repeat request (ARQ) can be used to establish that data is received without error | 3 hours | **1** Chapter 2 page 54<br>**2** Chapter 2 pages 55–59<br>**3** Chapter 2 pages 59–62<br>**4** Chapter 2 page 62 | **1** Workbook 1 Ch 2 question 3<br>**2** Workbook 1 Ch 2 questions 5–6, 8f–j, 9d,e,g, 10<br>**3** Workbook 1 Ch 2 questions 7, 9f<br>**4** Workbook 1 Ch 2 question 10 | |
| 2.3 Encryption | **1** The need for and purpose of encryption when transmitting data<br>**2** Symmetric and asymmetric encryption | 1 hour | **1** Chapter 2 page 63<br>**2** Chapter 2 pages 63–67 | **3** Workbook 1 Ch 2 questions 8a–e, 10<br>**4** Workbook 1 Ch 2 questions 9h, 10, 11 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 3** | **Hardware** | **22 hours** | | | |
| 3.1 Computer architecture | **1** The role of the central processing unit (CPU) in a computer; what is meant by a microprocessor<br>**2** The purpose of the components in a CPU, in a computer that has a von Neumann architecture; the role of each component in the process of the fetch–decode–execute cycle<br>**3** The meaning of CPU core, cache and clock, and how they can affect the performance of a CPU<br>**4** The purpose and use of a CPU instruction set<br>**5** The purpose and characteristics of embedded systems and the devices in which they are commonly used | 6 hours | **1** Chapter 3 page 75<br>**2** Chapter 3 pages 75–80<br>**3** Chapter 3 pages 80–82<br>**4** Chapter 3 page 82<br>**5** Chapter 3 pages 83–87 | **2** Workbook 1 Ch 3 questions 1–2<br>**3** Workbook 1 Ch 3 question 3<br>**4** Workbook 1 Ch 3 question 10<br>**5** Workbook 1 Ch 3 question 4 | Use pictures, diagrams and any computers that can be taken apart to show the components. |
| 3.2 Input and output devices | **1** Input devices and why they are required<br>**2** Output devices and why they are required<br>**3** Sensors including why they are required; the type of data captured by each sensor and when it would be used; selecting the most suitable sensor for a given context | 8 hours | **1** Chapter 3 pages 88–101<br>**2** Chapter 3 pages 101–110<br>**3** Chapter 3 pages 111–118 | **1** Workbook 1 Ch 3 questions 5–8<br>**2** Workbook 1 Ch 3 questions 9–11<br>**3** Workbook 1 Ch 3 questions 12–13 | Use any devices that are available to show how they work. |
| 3.3 Data storage | **1** Primary storage<br>**2** Secondary storage<br>**3** The operation and examples of magnetic, optical and solid-state storage<br>**4** Virtual memory, how it is created and used and why it is necessary<br>**5** Cloud storage<br>**6** The advantages and disadvantages of storing data on the cloud | 4 hours | **1** Chapter 3 pages 120–123<br>**2** Chapter 3 page 123<br>**3** Chapter 3 pages 123–128<br>**4** Chapter 3 pages 128–130<br>**5** Chapter 3 page 130<br>**6** Chapter 3 page 131 | **1** Workbook 1 Ch 3 question 14<br>**2** Workbook 1 Ch 3 question 14<br>**3** Workbook 1 Ch 3 questions 15, 16<br>**4** Workbook 1 Ch 3 question 17<br>**5** Workbook 1 Ch 3 questions 18a,b, 19<br>**6** Workbook 1 Ch 3 question 18c | |
| 3.4 Network hardware | **1** The need for and use of a network interface card (NIC)<br>**2** The purpose and structure of a media access control (MAC) address<br>**3** The purpose of an internet protocol (IP) address; the different types of IP address<br>**4** The role of a router in a network | 4 hours | **1** Chapter 3 page 133<br>**2** Chapter 3 page 133<br>**3** Chapter 3 pages 134–136<br>**4** Chapter 3 pages 136–137 Activity page 137 | **1** Workbook 1 Ch 3 question 20a<br>**2** Workbook 1 Ch 3 question 20a<br>**3** Workbook 1 Ch 3 questions 20a,b, 21<br>**4** Workbook 1 Ch 3 questions 20a,c | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 8** | **Programming** | **30 hours** | | | |
| 8.1 Programming concepts | **1** Declare and use variables and constants<br>**2** Basic data types<br>**3** Input and output<br>**4** Sequence; selection; iteration; totalling; counting; string handling; and arithmetic, logical and Boolean operators<br>**5** Nested statements<br>**6** Procedures, functions and parameters; local and global variables<br>**7** Library routines<br>**8** Creation of a maintainable program | 25 hours | **1** Chapter 8 pages 302–303<br>**2** Chapter 8 page 304<br>**3** Chapter 8 pages 304–307<br>**4** Chapter 8 pages 307–318<br>**5** Chapter 8 pages 318–321<br>**6** Chapter 8 pages 321–327<br>**7** Chapter 8 pages 327–328<br>**8** Chapter 8 pages 328–329 | **1** Workbook 2 Ch 8 question 2d<br>**2** Workbook 2 Ch 8 question 1<br>**3** Workbook 2 Ch 8 question 2<br>**4** Workbook 2 Ch 8 questions 3, 4, 7, 8<br>**5** Workbook 2 Ch 8 questions 8–9<br>**6** Workbook 2 Ch 8 question 10<br>**7** Workbook 2 Ch 8 question 11<br>**8** Workbook 2 Ch 8 questions 5, 6, 14, 15 | Decide on the IDE and programming language to use before starting to teach this section. There is further information on this in Workbook 2 Chapter 8. |
| 8.2 Arrays | **1** One-dimensional and two-dimensional arrays<br>**2** Use of arrays<br>**3** Writing and reading values into an array (indexing) using iteration | 3 hours | **1** Chapter 8 page 329<br>**2** Chapter 8 pages 329–333<br>**3** Chapter 8 pages 329–333 | **1** Workbook 2 Ch 8 question 12<br>**2** Workbook 2 Ch 8 question 12<br>**3** Workbook 2 Ch 8 questions 12, 14, 15 | Use of arrays, indexing and iteration are taught together. |
| 8.3 File handling | **1** The purpose of storing data in files to be used by a program<br>**2** Opening, closing and using a file for reading and writing | 2 hours | **1** Chapter 8 page 333<br>**2** Chapter 8 pages 333–336 | **1** Workbook 2 Ch 8 question 13a<br>**2** Workbook 2 Ch 8 question 13b,c | |
| **Topic 4** | **Software** | **8 hours** | | | |
| 4.1 Types of software and interrupts | **1** The difference between system software and application software and examples of each<br>**2** The role and basic functions of an operating system<br>**3** How hardware, firmware and an operating system are required to run applications software<br>**4** The role and operation of interrupts | 5 hours | **1** Chapter 4 pages 147–155<br>**2** Chapter 4 pages 155–160<br>**3** Chapter 4 pages 160–161<br>**4** Chapter 4 pages 161–163 Activity page 164 | **1** Workbook 1 Ch 4 questions 1–3<br>**2** Workbook 1 Ch 4 questions 4, 5, 7, 8<br>**3** Workbook 1 Ch 4 question 9<br>**4** Workbook 1 Ch 4 question 6 | |
| 4.2 Types of programming language, translators and integrated development environments (IDEs) | **1** High- and low-level languages, including the advantages and disadvantages of each<br>**2** Assembly language and the need for assemblers<br>**3** The operation of a compiler and an interpreter, including how a high-level language is translated by each and how errors are reported<br>**4** Advantages and disadvantages of a compiler and an interpreter<br>**5** The role of an IDE and the common functions it provides | 3 hours | **1** Chapter 4 pages 166–167<br>**2** Chapter 4 pages 167–168<br>**3** Chapter 4 pages 168–169<br>**4** Chapter 4 page 170<br>**5** Chapter 4 pages 170–173 Activity page 173 | **1** Workbook 1 Ch 4 question 10<br>**2** Workbook 1 Ch 4 questions 11, 12c<br>**3** Workbook 1 Ch 4 questions 11, 12a,b<br>**4** Workbook 1 Ch 4 question 13<br>**5** Workbook 1 Ch 4 question 14 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 5** | **The internet and cyber security** | **8 hours** | | | |
| 5.1 The internet and the world wide web | **1** The difference between the internet and the world wide web<br>**2** The meaning of a uniform resource locator (URL)<br>**3** The purpose and operation of hypertext transfer protocol (HTTP) and hypertext transfer protocol secure (HTTPS)<br>**4** The purpose and functions of a web browser<br>**5** The location, retrieval and display of web pages on a device when a user enters a URL<br>**6** Cookies and how they are used, including session cookies and persistent cookies | 6 hours | **1** Chapter 5 pages 180–181<br>**2** Chapter 5 page 181<br>**3** Chapter 5 page 181<br>**4** Chapter 5 page 182<br>**5** Chapter 5 page 183<br>**6** Chapter 5 pages 184–186 Activity page 186 | **1** Workbook 1 Ch 5 question 1a<br>**2** Workbook 1 Ch 5 question 1b<br>**3** Workbook 1 Ch 5 question 1a<br>**4** Workbook 1 Ch 5 question 1c<br>**5** Workbook 1 Ch 5 question 2<br>**6** Workbook 1 Ch 5 question 3 | |
| 5.2 Digital currency | **1** The concept of digital currency and how digital currencies are used<br>**2** The process of blockchain and how it is used to track digital currency | 1 hour | **1** Chapter 5 pages 186–187<br>**2** Chapter 5 pages 187–189 | **1** Workbook 1 Ch 5 question 4<br>**2** Workbook 1 Ch 5 question 4 | |
| 5.3 Cyber security | **1** The processes involved in and the aim of carrying out a range of cyber security threats<br>**2** The solutions used to help keep data safe from security threats | 1 hour | **1** Chapter 5 pages 189–198<br>**2** Chapter 5 pages 198–210 | **1** Workbook 1 Ch 5 questions 5–6<br>**2** Workbook 1 Ch 5 questions 7–11 | |
| **Topic 9** | **Databases** | **8 hours** | | | |
| | **1** Single-table database<br>**2** Basic data types for databases<br>**3** Primary key, including identifying a suitable primary key<br>**4** Use of SQL scripts to query data stored in a single-table database | 8 hours | **1** Chapter 9 pages 339–242<br>**2** Chapter 9 pages 342–343<br>**3** Chapter 9 pages 343–344<br>**4** Chapter 9 pages 344–351 | **1** Workbook 2 Ch 9 question 1a<br>**2** Workbook 2 Ch 9 questions 1b, 2a, 4a<br>**3** Workbook 2 Ch 9 questions 2b, 4a<br>**4** Workbook 2 Ch 9 questions 3, 4c,d | Practical use of a single-table database with SQL is strongly advised. |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 6** | **Automated and emerging technologies** | **6 hours** | | | |
| 6.1 Automated systems | **1** The use of sensors, microprocessors and actuators to create automated systems<br>**2** The advantages and disadvantages of automated systems used for given scenarios | 2 hours | **1** Chapter 6 pages 217–218<br>**2** Chapter 6 pages 218–229 | **1** Workbook 1 Ch 6 questions 3a,b,c–6, 4a,b, 5, 6a<br>**2** Workbook 1 Ch 6 questions 3d,e, 4c, 6b | |
| 6.2 Robotics | **1** Robotics<br>**2** The characteristics of a robot<br>**3** The roles that robots can perform and the advantages and disadvantages of their use | 2 hours | **1** Chapter 6 page 230<br>**2** Chapter 6 pages 231–232<br>**3** Chapter 6 pages 232–241 | **1,2,3** Workbook 1 Ch 6 question 2<br>**3** Workbook 1 Ch 6 question 10 | Use videos to demonstrate. |
| 6.3 Artificial intelligence | **1** Artificial intelligence (AI)<br>**2** The main characteristics of AI (the collection of data and the rules for using that data, the ability to reason, and can include the ability to learn and adapt)<br>**3** The basic operation and components of AI systems to simulate intelligent behaviour | 2 hours | **1** Chapter 6 page 241<br>**2** Chapter 6 pages 241–242<br>**3** Chapter 6 pages 243–250 | **1,2,3** Workbook 1 Ch 6 questions 2, 8, 9<br>**3** Workbook 1 Ch 6 questions 1, 7 | |
| **Topic 10** | **Boolean logic** | **6 hours** | | | |
| | **1** Standard symbols for logic gates<br>**2** Functions of the logic gates<br>**3** Use of logic gates to create given logic circuits; complete truth tables; write logic expressions | 6 hours | **1** Chapter 10 pages 356–358<br>**2** Chapter 10 pages 358–360<br>**3** Chapter 10 pages 360–378 | **1** Workbook 2 Ch 10 questions 3b, 7b, 8b, 9a, 11b, 12a<br>**2** Workbook 2 Ch 10 question 1<br>**3** Workbook 2 Ch 10 questions 2–12 | The standard logic gate symbols are shown on page 32 of the syllabus. |

# Suggested Scheme of Work 2

The following outline Scheme of Work is a potential scheme – you should adapt it to the needs of your students.

## Teaching in topic order

In the scheme of work, Workbook 1 refers to the Computer Systems Workbook, and Workbook 2 refers to the Algorithms, Programming and Logic Workbook.

The times given are approximate time allocations for each topic and sub-topic of the syllabus.

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 1** | **Data representation** | **12 hours** | | | |
| 1.1 Number systems | **1** How and why computers use binary to represent all forms of data<br>**2** Denary, binary and hexadecimal number systems and conversion of positive integers between these number systems<br>**3** How and why hexadecimal is used as a beneficial method of data representation<br>**4** Addition of two positive 8-bit binary integers and the concept of overflow and why it occurs in binary addition<br>**5** Logical binary shifts on positive 8-bit binary integers and the effect this has on the positive binary integer<br>**6** Two's complement to represent positive and negative 8-bit binary integers | 7 hours | **1** Chapter 1 pages 2–3<br>**2** Chapter 1 pages 3–12<br>**3** Chapter 1 pages 12–15<br>**4** Chapter 1 pages 15–17<br>**5** Chapter 1 pages 17–20<br>**6** Chapter 1 pages 20–25 | **2** Workbook 1, Ch 1, questions 1–6, 8–9, 12a, 13a, 23<br>**3** Workbook 1, Ch 1, question 7<br>**4** Workbook 1, Ch 1, questions 10–11, 13b, 19<br>**5** Workbook 1, Ch 1, questions 12b,c–13c,d, 19<br>**6** Workbook 1, Ch 1, questions 14, 19 | It is essential to provide practice in conversion, addition, shifting and use of two's complement. There are plenty of online sources that provide extra practice. |
| 1.2 Text, sound and images | **1** How and why a computer represents text and the use of character sets, including ASCII and Unicode<br>**2** How and why a computer represents sound, including the effects of the sample rate and sample resolution<br>**3** How and why a computer represents an image, including the effects of the resolution and colour depth | 3 hours | **1** Chapter 1 pages 25–28<br>**2** Chapter 1 pages 29–30<br>**3** Chapter 1 pages 30–31 | **1** Workbook 1, Ch 1, question 18, 19<br>**2** Workbook 1, Ch 1, questions 19, 20b,c, 22<br>**3** Workbook 1, Ch 1, questions 17, 19, 20d | |
| 1.3 Data storage and file compression | **1** How data storage is measured<br>**2** Calculation of file sizes of image and sound files<br>**3** The purpose of and need for data compression<br>**4** Lossy and lossless file compression methods | 2 hours | **1** Chapter 1 page 32<br>**2** Chapter 1 pages 33–34<br>**3** Chapter 1 page 34<br>**4** Chapter 1 pages 34–37 | **1** Workbook 1, Ch 1, question 3<br>**2** Workbook 1, Ch 1, question 15<br>**3** Workbook 1, Ch 1, question 16a<br>**4** Workbook 1, Ch 1, questions 16b,c, 20e, 21 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 2** | **Data transmission** | **8 hours** | | | |
| 2.1 Types and methods of data transmission | **1** Breaking data into packets to be transmitted; the structure of a packet; the process of packet switching<br>**2** How data is transmitted from one device to another using different methods of data transmission; the suitability of each method of data transmission for a given scenario<br>**3** The universal serial bus (USB) interface and how it is used to transmit data | 4 hours | **1** Chapter 2 pages 45–49<br>**2** Chapter 2 pages 49–52<br>**3** Chapter 2 pages 52–53 | **1** Workbook 1 Ch 2 questions 1–2, 9a, 10<br>**2** Workbook 1 Ch 2 questions 3, 9b, 10<br>**3** Workbook 1 Ch 2 questions 4, 9c | There are useful online demonstrations available. |
| 2.2 Methods of error detection | **1** The need to check for errors after data transmission and how these errors can occur<br>**2** The processes involved in error detection methods for detecting errors in data after transmission: parity check (odd and even), checksum, echo check<br>**3** How a check digit is used to detect errors in data entry, including ISBNs and bar codes<br>**4** How an automatic repeat request (ARQ) can be used to establish that data is received without error | 3 hours | **1** Chapter 2 page 54<br>**2** Chapter 2 pages 55–59<br>**3** Chapter 2 pages 59–62<br>**4** Chapter 2 page 62 | **1** Workbook 1 Ch 2 question 3<br>**2** Workbook 1 Ch 2 questions 5–6, 8f–j, 9d,e,g, 10<br>**3** Workbook 1 Ch 2 questions 7, 9f<br>**4** Workbook 1 Ch 2 question 10 | |
| 2.3 Encryption | **1** The need for and purpose of encryption when transmitting data<br>**2** Symmetric and asymmetric encryption | 1 hour | **1** Chapter 2 page 63<br>**2** Chapter 2 pages 63–67 | **3** Workbook 1 Ch 2 questions 8a–e, 10<br>**4** Workbook 1 Ch 2 questions 9h, 10, 11 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 3** | **Hardware** | **22 hours** | | | |
| 3.1 Computer architecture | **1** The role of the central processing unit (CPU) in a computer; what is meant by a microprocessor<br>**2** The purpose of the components in a CPU, in a computer that has a von Neumann architecture; the role of each component in the process of the fetch–decode–execute cycle<br>**3** The meaning of CPU core, cache and clock, and how they can affect the performance of a CPU<br>**4** The purpose and use of a CPU instruction set<br>**5** The purpose and characteristics of embedded systems and the devices in which they are commonly used | 6 hours | **1** Chapter 3 page 75<br>**2** Chapter 3 pages 75–80<br>**3** Chapter 3 pages 80–82<br>**4** Chapter 3 page 82<br>**5** Chapter 3 pages 83–87 | **2** Workbook 1 Ch 3 questions 1–2<br>**3** Workbook 1 Ch 3 question 3<br>**4** Workbook 1 Ch 3 question 10<br>**5** Workbook 1 Ch 3 question 4 | Use pictures, diagrams and any computers that can be taken apart to show the components. |
| 3.2 Input and output devices | **1** Input devices and why they are required<br>**2** Output devices and why they are required<br>**3** Sensors, including why they are required; the type of data captured by each sensor and when it would be used; selecting the most suitable sensor for a given context | 8 hours | **1** Chapter 3 pages 88–101<br>**2** Chapter 3 pages 101–110<br>**3** Chapter 3 pages 111–118 | **1** Workbook 1 Ch 3 questions 5–8<br>**2** Workbook 1 Ch 3 questions 9–11<br>**3** Workbook 1 Ch 3 questions 12–13 | Use any devices that are available to show how they work. |
| 3.3 Data storage | **1** Primary storage<br>**2** Secondary storage<br>**3** The operation and examples of magnetic, optical and solid-state storage<br>**4** Virtual memory, how it is created and used and why it is necessary<br>**5** Cloud storage<br>**6** The advantages and disadvantages of storing data on the cloud | 4 hours | **1** Chapter 3 pages 120–123<br>**2** Chapter 3 page 123<br>**3** Chapter 3 pages 123–128<br>**4** Chapter 3 pages 128–130<br>**5** Chapter 3 page 130<br>**6** Chapter 3 page 131 | **1** Workbook 1 Ch 3 question 14<br>**2** Workbook 1 Ch 3 question 14<br>**3** Workbook 1 Ch 3 questions 15, 16<br>**4** Workbook 1 Ch 3 questions 19, 21<br>**5** Workbook 1 Ch 3 questions 18a,b, 19<br>**6** Workbook 1 Ch 3 question 18c | |
| 3.4 Network hardware | **1** The need for and use of a network interface card (NIC)<br>**2** The purpose and structure of a media access control (MAC) address<br>**3** The purpose of an internet protocol (IP) address; the different types of IP address<br>**4** The role of a router in a network | 4 hours | **1** Chapter 3 page 133<br>**2** Chapter 3 page 133<br>**3** Chapter 3 pages 134–136<br>**4** Chapter 3 pages 136–137<br>Activity page 137 | **1** Workbook 1 Ch 3 question 22a<br>**2** Workbook 1 Ch 3 question 22a<br>**3** Workbook 1 Ch 3 questions 20a,b, 21<br>**4** Workbook 1 Ch 3 questions 20a,c | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 4** | **Software** | **8 hours** | | | |
| 4.1 Types of software and interrupts | **1** The difference between system software and application software and examples of each<br>**2** The role and basic functions of an operating system<br>**3** How hardware, firmware and an operating system are required to run applications software<br>**4** The role and operation of interrupts | 5 hours | **1** Chapter 4 pages 147–155<br>**2** Chapter 4 pages 155–160<br>**3** Chapter 4 pages 160–161<br>**4** Chapter 4 pages 161–163 Activity page 164 | **1** Workbook 1 Ch 4 questions 1–3<br>**2** Workbook 1 Ch 4 questions 4, 5, 7, 8<br>**3** Workbook 1 Ch 4 question 9<br>**4** Workbook 1 Ch 4 question 6 | |
| 4.2 Types of programming language, translators and integrated development environments (IDEs) | **1** High- and low-level languages, including the advantages and disadvantages of each<br>**2** Assembly language and the need for assemblers<br>**3** The operation of a compiler and an interpreter, including how a high-level language is translated by each and how errors are reported<br>**4** Advantages and disadvantages of a compiler and an interpreter<br>**5** The role of an IDE and the common functions it provides | 3 hours | **1** Chapter 4 pages 166–167<br>**2** Chapter 4 pages 167–168<br>**3** Chapter 4 pages 168–169<br>**4** Chapter 4 page 170<br>**5** Chapter 4 pages 170–173 Activity page 173 | **1** Workbook 1 Ch 4 question 10<br>**2** Workbook 1 Ch 4 questions 11, 12c<br>**3** Workbook 1 Ch 4 questions 11, 12a,b<br>**4** Workbook 1 Ch 4 question 13<br>**5** Workbook 1 Ch 4 question 14 | |
| **Topic 5** | **The internet and cyber security** | **8 hours** | | | |
| 5.1 The internet and the world wide web | **1** The difference between the internet and the world wide web<br>**2** The meaning of a uniform resource locator (URL)<br>**3** The purpose and operation of hypertext transfer protocol (HTTP) and hypertext transfer protocol secure (HTTPS)<br>**4** The purpose and functions of a web browser<br>**5** The location, retrieval and display of web pages on a device when a user enters a URL<br>**6** Cookies and how they are used, including session cookies and persistent cookies | 6 hours | **1** Chapter 5 pages 180–181<br>**2** Chapter 5 page 181<br>**3** Chapter 5 page 181<br>**4** Chapter 5 page 182<br>**5** Chapter 5 page 183<br>**6** Chapter 5 pages 184–186 Activity page 186 | **1** Workbook 1 Ch 5 question 1a<br>**2** Workbook 1 Ch 5 question 1b<br>**3** Workbook 1 Ch 5 question 1a<br>**4** Workbook 1 Ch 5 question 1c<br>**5** Workbook 1 Ch 5 question 2<br>**6** Workbook 1 Ch 5 question 3 | |
| 5.2 Digital currency | **1** The concept of digital currency and how digital currencies are used<br>**2** The process of blockchain and how it is used to track digital currency | 1 hour | **1** Chapter 5 pages 186–187<br>**2** Chapter 5 pages 187–189 | **1** Workbook 1 Ch 5 question 4<br>**2** Workbook 1 Ch 5 question 4 | |
| 5.3 Cyber security | **1** The processes involved in and the aim of carrying out a range of cyber security threats<br>**2** The solutions used to help keep data safe from security threats | 1 hour | **1** Chapter 5 pages 189–198<br>**2** Chapter 5 pages 198–210 | **1** Workbook 1 Ch 5 questions 5–6<br>**2** Workbook 1 Ch 5 questions 7–11 | |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 6** | **Automated and emerging technologies** | **6 hours** | | | |
| 6.1 Automated systems | **1** The use of sensors, microprocessors and actuators to create automated systems<br>**2** The advantages and disadvantages of automated systems used for given scenarios | 2 hours | **1** Chapter 6 pages 217–218<br>**2** Chapter 6 pages 218–229 | **1** Workbook 1 Ch 6 questions 3a,b,c–6, 4a,b, 5, 6a<br>**2** Workbook 1 Ch 6 questions 3d,e, 4c, 6b | |
| 6.2 Robotics | **1** Robotics<br>**2** The characteristics of a robot<br>**3** The roles that robots can perform and the advantages and disadvantages of their use | 2 hours | **1** Chapter 6 page 230<br>**2** Chapter 6 pages 231–232<br>**3** Chapter 6 pages 232–241 | **1,2,3** Workbook 1 Ch 6 question 2<br>**3** Workbook 1 Ch 6 question 10 | Use videos to demonstrate. |
| 6.3 Artificial intelligence | **1** Artificial intelligence (AI)<br>**2** The main characteristics of AI (the collection of data and the rules for using that data, the ability to reason, and can include the ability to learn and adapt)<br>**3** The basic operation and components of AI systems to simulate intelligent behaviour | 2 hours | **1** Chapter 6 page 241<br>**2** Chapter 6 pages 241–242<br>**3** Chapter 6 pages 243–250 | **1,2,3** Workbook 1 Ch 6 questions 2, 8, 9<br>**3** Workbook 1 Ch 6 questions 1, 7 | |
| **Topic 7** | **Algorithm design and problem solving** | **21 hours** | | | |
| | **1** The program development life cycle: analysis, design, coding and testing<br>**2** Computer systems and sub-systems; problem decomposition; methods to design and construct a solution to a problem<br>**3** The purpose of a given algorithm<br>**4** Standard methods of solution<br>**5** The need for validation and verification checks on input data; different types of validation and verification checks<br>**6** Use of test data<br>**7** Use of trace tables to document dry-runs of algorithms<br>**8** Identification and correction of errors in algorithms<br>**9** Writing and amending algorithms for given problems/scenarios, using pseudocode, program code and flowcharts | 21 hours | **1** Chapter 7 pages 258–259<br>**2** Chapter 7 page 271<br>**3** Chapter 7 pages 271–272<br>**4** Chapter 7 pages 272–276<br>**5** Chapter 7 pages 276–281<br>**6** Chapter 7 pages 281–282<br>**7** Chapter 7 pages 282–285<br>**8** Chapter 7 pages 285–288<br>**9** Chapter 7 pages 288–292 | **1** Workbook 2 Ch 7 question 1<br>**2** Workbook 2 Ch 7 questions 2–8<br>**3** Workbook 2 Ch 7 questions 9, 10<br>**4** Workbook 2 Ch 7 question 12<br>**5** Workbook 2 Ch 7 question 13<br>**6** Workbook 2 Ch 7 question 14<br>**7** Workbook 2 Ch 7 question 15<br>**8** Workbook 2 Ch 7 question 16<br>**9** Workbook 2 Ch 7 question 15d | The flowchart symbols to use are on page 21 of the syllabus.<br>The pseudocode used is set out on pages 33–46 of the syllabus. |

| Syllabus reference | Content | Time | Student's Book pages | Workbook questions | Notes |
|---|---|---|---|---|---|
| **Topic 8** | **Programming** | **30 hours** | | | |
| 8.1 Programming concepts | **1** Declare and use variables and constants<br>**2** Basic data types<br>**3** Input and output<br>**4** Sequence; selection; iteration; totalling; counting; string handling; and arithmetic, logical and Boolean operators<br>**5** Nested statements<br>**6** Procedures, functions and parameters; local and global variables<br>**7** Library routines<br>**8** Creation of a maintainable program | 25 hours | **1** Chapter 8 pages 302–303<br>**2** Chapter 8 page 304<br>**3** Chapter 8 pages 304–307<br>**4** Chapter 8 pages 307–318<br>**5** Chapter 8 pages 318–321<br>**6** Chapter 8 pages 321–327<br>**7** Chapter 8 pages 327–328<br>**8** Chapter 8 pages 328–329 | **1** Workbook 2 Ch 8 question 2d<br>**2** Workbook 2 Ch 8 question 1<br>**3** Workbook 2 Ch 8 question 2<br>**4** Workbook 2 Ch 8 questions 3, 4, 7, 8<br>**5** Workbook 2 Ch 8 questions 8–9<br>**6** Workbook 2 Ch 8 question 10<br>**7** Workbook 2 Ch 8 question 11<br>**8** Workbook 2 Ch 8 questions 5, 6, 14, 15 | Decide on the IDE and programming language to use before starting to teach this section. There is further information on this in Workbook 2 Chapter 8. |
| 8.2 Arrays | **1** One-dimensional and two-dimensional arrays<br>**2** Use of arrays<br>**3** Writing and reading values into an array (indexing) using iteration | 3 hours | **1** Chapter 8 page 329<br>**2** Chapter 8 pages 329–333<br>**3** Chapter 8 pages 329–333 | **1** Workbook 2 Ch 8 question 12<br>**2** Workbook 2 Ch 8 question 12<br>**3** Workbook 2 Ch 8 questions 12, 14, 15 | Use, indexing and iteration are taught together. |
| 8.3 File handling | **1** The purpose of storing data in files to be used by a program<br>**2** Opening, closing and using a file for reading and writing | 2 hours | **1** Chapter 8 page 333<br>**2** Chapter 8 pages 333–336 | **1** Workbook 2 Ch 8 question 13a<br>**2** Workbook 2 Ch 8 question 13b,c | |
| **Topic 9** | **Databases** | **8 hours** | | | |
| | **1** Single-table database<br>**2** Basic data types for databases<br>**3** Primary key, including identifying a suitable primary key<br>**4** Use of SQL scripts to query data stored in a single-table database | 8 hours | **1** Chapter 9 pages 339–342<br>**2** Chapter 9 pages 342–343<br>**3** Chapter 9 pages 343–344<br>**4** Chapter 9 pages 344–351 | **1** Workbook 2 Ch 9 question 1a<br>**2** Workbook 2 Ch 9 questions 1b, 2a, 4a<br>**3** Workbook 2 Ch 9 questions 2b, 4a<br>**4** Workbook 2 Ch 9 questions 3, 4c,d | Practical use of a single-table database with SQL is strongly advised. |
| **Topic 10** | **Boolean logic** | **6 hours** | | | |
| | **1** Standard symbols for logic gates<br>**2** Functions of the logic gates<br>**3** Use of logic gates to create given logic circuits; complete truth tables; write logic expressions | 6 hours | **1** Chapter 10 pages 356–358<br>**2** Chapter 10 pages 358–360<br>**3** Chapter 10 pages 360–378 | **1** Workbook 2 Ch 10 questions 3b, 7b, 8b, 9a, 11b, 12a<br>**2** Workbook 2 Ch 10 question 1<br>**3** Workbook 2 Ch 10 questions 2–12 | The standard logic gate symbols are shown on page 32 of the syllabus. |

## Lesson plan template

**Topic**

Syllabus reference and content:

Learning objectives:

**Resources**

Textbook:

Online:

Specially prepared:

New vocabulary:

**Timed activities**

Lesson opener:

Group activity:

Individual activity:

Plenary:

**Homework**

**Reflection**

What went well:

What could be improved:

## How to use the lesson plan template

List the name of the topic and the section, page and content of the syllabus that will be covered in the lesson.
*Tip: always make sure that you use the version of the syllabus for the year your students sit the exam.*

List the learning objectives for the lesson.
*Tip: always make sure that you have these learning objectives clearly visible, e.g. on the whiteboard.*

# Lesson plan

**Topic**

Syllabus reference and content:

Learning objectives:

**Resources**

Textbook:

Online:

Specially prepared:

New vocabulary:

**Timed activities**

Lesson opener:

Group activity:

Individual activity:

Plenary:

**Homework**

**Reflection**

What went well:

What could be improved:

Provide references to the page(s) and section of the textbook and URLs for any online resources that you plan to use. Include details/links to any resources that you have prepared for this topic.
*Tip: always check all your resources just before the lesson as the addresses and availability of online resources can change without warning.*

List new technical terms to introduce them and their meanings.
*Tip: use the key terms list at the end of the chapter.*

Divide your lesson into three or four timed sections. Always include an introduction and a plenary. Try to include activities that promote active learning to engage your students throughout the lesson.
*Tip: always make sure that you include the learning objectives for the lesson in your introduction.*

The plenary section should include a summary of what should have been learned, homework if required and plans for the next lesson.
*Tip: the use of quick quizzes and gapped handouts can help re-enforce what has been learned in the lesson during the plenary.*

Always take time to reflect on how well each lesson delivered the learning objectives. This helps you to develop as a teacher.
*Tip: try to do this as close to delivering the lesson as you can.*

From time to time, ask your students how much they learned during a lesson.
*Tip: have the courage to include the opinions of your students in your reflections, as this can be really helpful.*

Set homework with clear instructions as to what is required, when it is to be handed in and how you expect to receive it.
*Tip: there are end-of-chapter questions in the Student's Book and questions in the workbooks that can be used for student homework.*

# Sample lesson plan 1

**Topic** Digital currency

Syllabus reference and content:

Section 5.2 (page 20)

1. Understand the concept of a digital currency and how digital currencies are used.

2. Understand the process of blockchain and how it is used to track digital currency transactions.

Notes and guidance

- A digital currency is one that only exists electronically.
- Blockchain is a digital ledger, that is, a time-stamped series of records that cannot be altered.

Learning objectives:

Understand what digital currency is and how it is used, including the use of blockchain.

**Resources**

Textbook: Chapter 5, section 5.2 (pages 186–189)

Online: https://youtu.be/SzAuB2FG79A (cryptocurrency) and https://youtu.be/3xGLc-zz9cA (blockchain)

Specially prepared: create a presentation on digital currency for plenary. Use the key terms on page 212.

New vocabulary: the key terms on page 212: digital currency, cryptocurrency, blockchain, time stamp and proof-of-work

**Timed activities (45-minute lesson)**

Lesson opener (5 mins): introduce the concept of digital currency and lead a short class discussion on what students know about the concept. Introduce the learning objectives.

Group activity (15 mins): play the first video on cryptocurrency. Pause and discuss it with the class. Then play the second video on blockchain. Pause and discuss it with the class.

Individual activity (15 mins): instruct students to read the section on how blockchain works (page 188) and then answer question 1 from Activity 5.2 (page 189).

Plenary (10 mins): create a presentation to check the class's understanding of new terminology and learning objectives. Look forward to the next lesson on cyber security. Set homework.

**Homework**

Students should complete Activity 5.1 (page 186) and the end-of-chapter question 5c (page 215).

They should record their answers in Microsoft Word and upload the file to the class Google Drive before the next lesson.

**Reflection:**

What went well:

Complete this after the lesson.

What could be improved:

Complete this after the lesson.

# Sample lesson plan 2

**Topic** Programming concepts: iteration

Syllabus reference and content:

Sections 8.1.4c and 8.1.4d (page 25)

1. Understand and use the concept of iteration, including count-controlled loops pre-condition loops and post-condition loops.
2. Understand and use the concepts of totalling and counting.

Learning objectives:

Understand and be able write programs using iteration, totalling and counting.

**Resources**

Textbook: Chapter 8, sections 8.1.4c and 8.1.4d (pages 312–314)

Specially prepared: three (two for Python) short programs to demonstrate for your chosen programming language: FOR loop (Table 10), pre-condition (Table 8.11) and post-condition (Table 8.12). Two short programs to demonstrate for your chosen programming language: totalling (Table 13) and counting (Table 8.14).

Create a presentation to show two or three short scenarios to use during the plenary so your students can decide which type of loop would be the most suitable to use. For example: a range check is required to check if a number input is positive and less than or equal to 100, outputting a list of 100 numbers. Additionally, include scenarios so students can decide whether to use counting or totalling, for example, finding out how many students in a class are aged 14 or finding the average age of the students in a class.

New vocabulary: key terms on page 336: iteration, counting, totalling, pre-condition loop, post-condition loop and count controlled loop.

**Timed activities (45-minute lesson)**

Lesson opener (15 mins): revisit the pseudocode for iteration with count-controlled loops FOR … NEXT, pre-condition loops DO WHILE … ENDWHILE and post-condition loops REPEAT … UNTIL. The details for this are on pages 268–270 of the Student's Book. Demonstrate how each of these loops can be programmed. Discuss why condition-controlled loops are used and the differences between pre- and post-condition. Demonstrate how counting and totalling are used and explain the difference.

Individual and paired activity (15 mins): instruct students to complete Activity 8.6 (page 314). Then split the class into pairs. Each student should test their partner's program, using trace tables to record their answers.

Plenary (15 mins): create a presentation with scenarios and terminology to check the class's understanding of new terminology and learning objectives. Look forward to next lesson on string handling. Set homework.

**Homework**

Students should complete Activity 8.7 (page 314) in the Student's Book. They should upload their work to the class's Google Drive before the next lesson.

**Reflection:**

What went well:

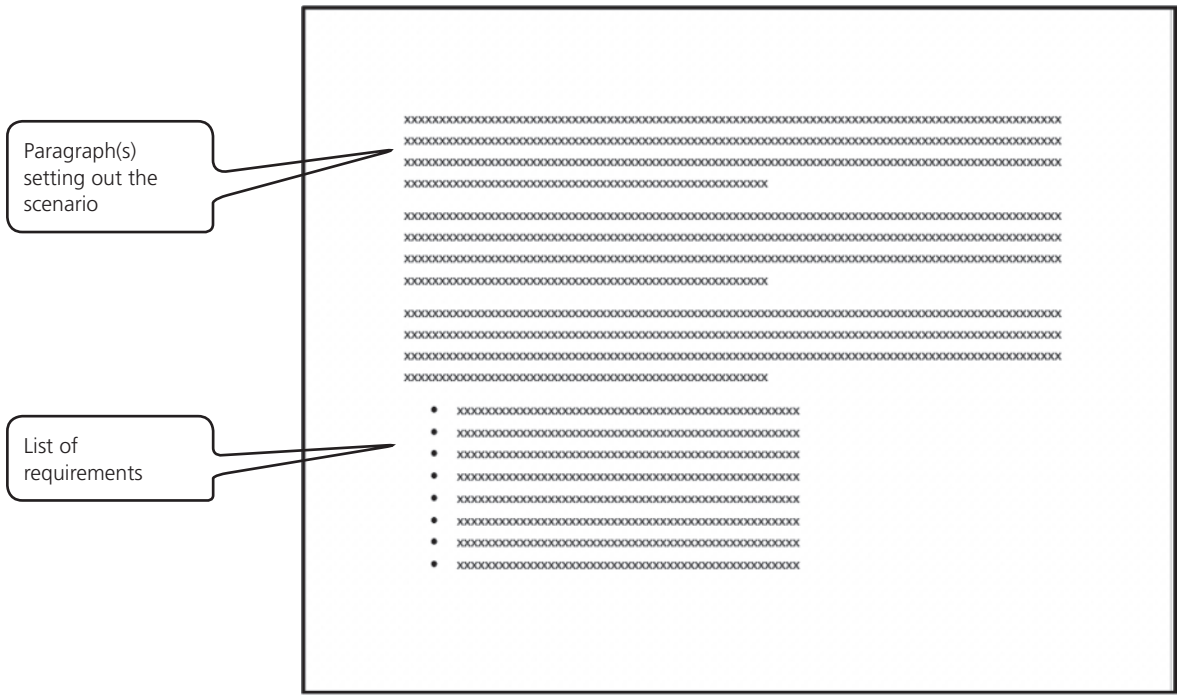
Complete this after the lesson.

What could be improved:

Complete this after the lesson.

# Scenario-based questions

## Scenario question structure




**How to approach a scenario question**

1 Read the scenario and the requirements through very carefully at least twice.

2 Make a note of any identifier names and their data structures mentioned in the scenario and remember to use those names and data structures when you write the program code or pseudocode for your answer.

> For example, a 2D array containing the names of students in a school Student[].
>
> Data structure mentioned
>
> Identifier name, shown in the question as Courier font

Other data structures needed should be given meaningful names.

3 Check the scenario for any special instructions about data structures.

> For example, the arrays have already been set up and the data stored.

4 Check the scenario for any special instructions about the program.

> For example, use a function to calculate the average age of the students in the school.

5 Decide how the program is to be written using the programming language that has been taught (Python, Visual Basic or Java)

**or**

using pseudocode.

6 Identify the programming techniques needed for each requirement.

> For example, a requirement to search for a student's name would require:
>
> Input of name
>
> Linear search of list
>
> Output whether found or not

7 Write code for each requirement in the list. For each requirement, check that everything has been completed to match the scenario and there are no logical errors. Include comments in the code to explain how the code works.

8 Finally read through your code to check that the program is in a logical order and all the requirements have been met.

9 Use a checklist to ensure you have completed everything.

**Suggested checklist**

| | DO | DON'T |
|---|---|---|
| Programming code or pseudocode | Choose a single programming language (Python, Visual Basic or Java) or pseudocode | Mix languages or mix pseudocode with a language |
| Fully commented solution | Use the appropriate commenting feature of the language chosen | Write notes all over the code<br>Write code without comments |
| All data structures required included using any identifier names given in the question and meaningful names for any others | Make sure that you use any names given exactly<br>Make sure that all the data required is stored appropriately | Change names used during the program<br>Add extra data structures that are not required |
| Code to fulfil all requirements included in a logical order | Check that code has been written to meet every part of every requirement<br>Check that there are no logical errors in the code<br>Check that the order of the code in the program is logical | Add extra code for other things that you would like the program to do |
| Appropriate techniques have been used, including any specifically mentioned in the scenario | Check that any special instructions about programming have been followed<br>Check that a suitable range of techniques have been used | Include extra techniques that are not required, e.g. a bubble sort if not part of the requirements |

# ESL guidance

## Teaching learners with English as a second language (ESL)

As it is an international qualification taught in many different countries around the world, the Cambridge IGCSE, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) are often taught to learners who do not speak English as a first language. This introduction will highlight some aspects of language difficulties that learners may have and suggest strategies to address them. There is also an accompanying set of language development activities, available on **boost-learning.com**.

There are some general principles that are likely to support all your learners but which will provide extra support for any learners with language needs.

### Learning objectives

When defining learning objectives for your lessons, give thought to the kind of language that learners will need in order to successfully participate in the lesson and use the knowledge that you plan to teach. Make sure that you define a language objective – this will help you plan appropriate activities and assessment for learners. If you are teaching a monolingual class, it will also help you plan when to allow learners to use their first language in lessons.

### Contextualise

Take the opportunity at the start of every lesson to link what is to be presented to what has already been taught and what learners already know. It is quite possible that some learners will already have some experience with some of the topics covered, so finding out what they already know is helpful. Giving second-language speakers of English this chance to think about the topic before starting the lesson will make it easier for them to understand the language used.

### Managing input

How will information be presented to the class? Are there ways of making the information more accessible for learners whose first language is not English? Consider the use of transcripts, background reading, amount of visual support and how long each piece of input lasts. If learners are listening or reading in their second language, they are likely to benefit from shorter sections of input interspersed with questions, discussions and collaborative tasks.

### Collaborative learning

This is when learners are obliged to work together and communicate with each other. Working collaboratively means that learners have to put new language learned to use, consolidating it in their memory. They have to use information that they have listened to or read in the lesson. This provides opportunities for the teacher to see any gaps in their knowledge or understanding. It also provides learners with valuable opportunities to communicate in English, which will help their overall language development.

### Assessment for learning

Regular formative assessment, learning checks and developmental feedback are particularly important when teaching learners who do not have English as a first language. Formative assessment allows teachers to check how well learners have understood and can express lesson content. It also helps learners to know which aspects of their language skills require development.

# Teaching vocabulary to ESL learners

Computer science is a subject with a considerable specialist vocabulary, and a need for precision. It requires a level of general academic vocabulary in addition to a wide knowledge of computer science terminology.

## The vocabulary of computer science

Like most subjects, computer science has its own vocabulary. This can be broken down into different categories, which present different challenges.

### Specific concepts

Much of the vocabulary that learners of computer science have to learn refers to specific concepts which are denoted by specific terms. For example, specific terms that occur in the Data representation topic include *character set*, *logical shift*, *overflow error*, *sampling resolution*, *Unicode*, etc.

This specialist vocabulary could potentially cause difficulties for learners who have English as a second language. They are unlikely to have learned specialist computer science terms in their general language lessons. The terminology used with higher-level computer science concepts may also be unknown to native English speakers, and so this language will be taught explicitly to learners as a matter of course. However, teachers should check that learners with English as a second language know the language used to express more basic concepts.

## Vocabulary difficulties

The syllabuses also require learners to be able to write definitions for specialist concepts. This requires precision, as well as sufficient vocabulary to be able to write the definitions without using the terms to be defined.

## Strategies for developing learners' computer science vocabulary

### Principles

There are a number of principles to consider when looking for effective ways of developing vocabulary.

Firstly, the frequency with which learners encounter new vocabulary is crucial in ensuring that they are able to easily recognise, recall and use vocabulary items. Therefore, teachers should ensure that learners encounter each new vocabulary item on numerous occasions. Research has shown that, in order to learn new vocabulary, learners need to encounter it seven to nine times on average. This makes opportunities for discussing content in class very valuable.

The second principle is that new vocabulary should also be presented to learners in context – this helps to make the meaning clear, and also gives a more memorable frame to support recall.

Thirdly, it has been suggested that the depth to which learners engage with new vocabulary increases their ability to memorise it. Teachers should make sure that learners get opportunities to actively use new vocabulary.

### Teacher strategies

Presenting new vocabulary in context is unlikely to be difficult in higher secondary classes. Learners should be encouraged to actively identify useful new vocabulary items whenever they are listening to lessons or reading texts about computer science.

It is also vital for teachers to build in as much reviewing and recapping of terminology as possible. Time should be spent at the start of each lesson reviewing key concepts that have been presented in recent lessons and encouraging learners to recall the terms themselves. There are various

engaging activities that teachers can use for this. Regular formative assessment will increase the frequency with which learners review these concepts. Online quiz applications such as *Kahoot* and *Quizlet* also provide opportunities for further practice. These types of reviews are very suitable for homework assignments. As learners often confuse similar terminology, it is beneficial to spend some time getting learners to work specifically on identifying the differences between related terms as well as distinguishing between them.

It is important to ensure that learners engage with the terms presented and have opportunities to use them actively as well as passively. Teachers should build in pair and group work into lessons, to ensure that learners have the chance to use new terms when communicating with each other. This is likely to lead to deeper engagement with new vocabulary.

### Learner strategies

Learners should be encouraged to keep records of new vocabulary. There are a number of ways that they can do this, for example, using language journals, word cards or shared online documents. These records should also contain information about related terms, how these terms differ and how they can be used. As well as keeping records, learners could also form study groups in which they talk through the topics studied, ensuring that they use new vocabulary. Learners should also be encouraged to actively practise recalling language from their records. This could be done using word games or quizzes similar to those used in class. To maximise their opportunities to find new vocabulary in context, it is beneficial for learners to expose themselves to as much computer science language as possible outside class.

## Developing ESL learners' reading skills for computer science

### Reading in Upper Secondary Computer Science

Computer science is a subject that requires learners to read in depth about a wide range of topics. Learners have to be able to read documents using general and specialist language and also be able to interpret diagrams, tables and images. Second-language speakers of English may have some of these skills in their first language, but may need to practise their skills in English. While they may have read about some topics relevant to computer science in their English language classes, this is unlikely to have been at the same level of detail and they may not have used such a wide range of ways to communicate information (particularly information presented in diagrams and tables). An additional problem that learners may have is that, if they have grown up in a different country, they may not know all the background information that texts assume of them.

### Reading skills

Reading is often considered to be made up of two different types of skills: bottom-up skills, which come from understanding the meaning of the words and structures on the page, and top-down skills which come from the reader's own understanding of the topic and situation described in a particular word problem. Successful reading involves the interaction of these two different sets of skills.

Consider, as an example, Chapter 5 in the Student's Book. In addition to the main text, the chapter contains images, diagrams, tables, tasks, exercises and exam-style questions. The text itself is divided by headings.

### Bottom-up skills

The previous section detailed the challenges that specialist vocabulary can present for second-language speakers of English (see page 26). Unsurprisingly, vocabulary knowledge is a very important part of being able to read successfully. In Chapter 5 of the Student's Book,

learners need to understand specialist terms such as *brute force attack*, *denial of service attack*, *packet sniffing*, *secure sockets layer* and *SSL certificate*. There are also terms that are not specific to computer science, but are much more common in formal and academic writing than in the everyday contexts that second-language learners are likely to be familiar with, such as *interconnect*, *infrastructure, convert*, *associated*, *retrieve* and *terminate*.

As well as knowledge of whole words, learners may be able to use their knowledge of prefixes and suffixes to work out the meaning of new words. For example, a learner who recognises the similarity between the words *anti-spyware* and *anti-virus* will be able to work out the meaning.

### Top-down skills

As well as knowledge of the text, successful readers use top-down knowledge from outside the text. In most chapters, readers can use their own knowledge of computer science topics to predict what the text will say. As they read, they will check to see whether (and how) the text adds to their previous understanding or whether it challenges this understanding in any way. Readers are likely to use features such as images and headings to support these predictions: they will read the headings or look at images and consider what they think the text will say.

Readers of computer science textbooks will understand how information is presented in different ways and will have an understanding of when to read through the text, and when to look at diagrams, equations or sections of code and relate them to the concepts in the text. Where information is presented separately, such as in the Advice boxes, effective readers will know when to break off from reading the text and then how to relate the information back to the main text.

The way in which top-down and bottom-up skills interact will depend on the learners' purpose when reading the text. For example, they might be looking for specific information or trying to gain a general view of a topic.

### Think out loud

Most of the reading that learners encounter in studying computer science is reading word problems. Word problems are short, concise and full of information that needs to be carefully 'unpacked'. Encourage learners to read word problems out loud, asking for help along the way with any terms they don't understand. When they have finished, ask them to explain the word problem in their own words and try to work it out.

## Strategies to support learners

Bottom-up reading skills are dependent on language knowledge, which can be developed using the strategies described in the vocabulary section above. Extensive practice in reading computer science problems will help learners consolidate this knowledge as well as identify new language items that they need to learn. Learners should be encouraged to exploit reading texts to find useful new vocabulary items. However, as well as developing learners' language knowledge, it is also important to support learners to use their top-down skills and become strategic readers.

### Consider what learners already know

When presenting a new text in class, teachers should ask learners what they already know about the topic. This helps learners to understand the purpose for reading the text – they can confirm, add to or refute their current understanding. This review of prior knowledge also provides an opportunity to recap key vocabulary related to the text.

### Use of KNL charts

Use a KNL (know, need, learned) chart like the one below to find out what students already know before starting a new chapter. Refer to this chart at different points in the chapter to make sure

that students are learning the correct information and are aware that they are learning it, and to ensure that you are building upon what they already know.

| I already know | I need to know | I've learned |
|---|---|---|
| | | |

### Use text features

Once learners are thinking about the subject of the text, features in the text can be used to get learners to predict what they will find when they read it. Learners should look at the title, subheadings and diagrams or tables in order to inform their predictions.

### Close reading

On subsequent readings of the word problem, learners could be asked to identify any command words, relevant and irrelevant information, and language indicating the need for the use of specific processes.

If learners do not know the meaning of a word or phrase, ask questions to support them to work it out rather than giving them the answer straightaway – it is important to encourage learners to develop strategies for doing this independently in the future. Learners should be encouraged to check their understanding of the problem as they go through it with both their real-world knowledge of the context to check that it makes logical sense and with their knowledge of computers and computer science.

### Vocabulary focus

To help learners develop their vocabulary knowledge, they could be asked to find words in the word problem that fit particular definitions. They could be encouraged to find synonyms in the word problem. They could also be asked to complete gapped sentences with words from the word problem.

### Comprehension focus

Questions that check learners' comprehension of the word problem will give them a focus for reading. For example, students could answer direct questions, identify true and false sentences or complete an activity such as listing all the steps of a process in a flow chart. This type of activity will also provide teachers with an opportunity to assess learners' reading skills.

### Talking through texts

Another useful technique is to ask learners to talk through problems as they read them. This gives them the opportunity to see how problems and processes are linked and provides guidance as to how they can approach these problems in future. This may be a particularly useful technique as part of formative assessment, allowing teachers to make an accurate judgement of a learner's knowledge of computer science without this judgement being affected by the learner's English language level.

### Notetaking practice

Learners can be trained to take notes more effectively: they should set themselves a purpose before reading and note any important new vocabulary to look up and any questions that the word problem raises. One way these skills can be developed is to provide learners with different templates that support different note-taking strategies. Learners could also practise taking notes in class from short extracts, and then compare them with the notes taken by their classmates.

### Concept maps

An excellent way to review key terms covered in a chapter is to write them on the board. Students choose any two words and explain how they are related or make a sentence that includes both words.

### Use of visuals and graphic organisers

Many computer science concepts can be effectively communicated by using either visuals or graphic organisers. Teachers can use these in two ways. Firstly, provide the class with a visual or graphic organiser and ask them to label the parts and fill in key information in words. Then they can discuss and compare their work with a partner. Finally, students can use the completed visual as a prompt to write an explanation or description.

## Developing listening skills with ESL learners

Listening in lessons is clearly an important skill for learners of computer science and can be challenging for learners who have English as a second language.

### Listening skills

As with reading skills, teachers should consider the interaction between different kinds of skills and knowledge that students use when listening in class.

### Bottom-up skills

In order to understand a lesson, learners need to be able to use knowledge of language. When listening, there are two aspects to this.

Firstly, learners need to have sufficient knowledge of the grammar and vocabulary that will feature in the lesson. As we have seen, a considerable amount of the vocabulary is specific to computer science.

Secondly, learners have to be able to decode the words spoken from the stream of sound that they hear. This can be challenging because spoken language does not always closely reflect the written form; gaps in the stream of speech do not fall in the same place as gaps between words in a written text. The following are examples of types of phrases that might confuse learners. Learners may hear the phrase *free to use* as *free ta use*. Learners may hear letters that are not in the written words – they might hear a *w* sound between the words in a phrase such as *to enter*. Many words have different pronunciations depending on their position and significance in a sentence. For example, *of* may be pronounced more like *uv* in some phrases: *layers of an object* might sound like *layer zuv an object*.

### Top-down skills

In addition to being able to decode spoken sound, listeners also use background knowledge to build understanding while they are listening. In computer science lessons, learners will use their knowledge of computer science concepts to predict what will be described and to check their understanding of the lesson. Effective listeners will also use their understanding of the format of a lesson and what they expect the teacher to do to support their understanding of what is being described.

### Language development activities

Dictation activities can be particularly useful in helping learners develop their ability to recognise words in a stream of English speech. The aim of dictation activities is to raise learners' awareness of features of connected speech in English. Recognition of these features will make it easier for

learners to understand spoken English, and so it is best to focus on these aspects when correcting learners' dictations, rather than on aspects such as spelling or grammar. As well as dictation activities, it is also useful to give learners a more structured focus to a listening task.

### Key words

Dictate a few lines of a word problem from the Student's Book that are rich in computer science specialist terms, and ask students to identify and record the terms. Draw students' attention to the words that these terms combine with to form common phrases (which are known as collocations). Also point out the verbs that are normally used with the specialist terms. Once students become familiar with how technical terms combine with other words and which verbs are normally used with these terms, they will be more likely to use the specialist vocabulary in their own speech and writing.

# Glossary

**abnormal data** data that is rejected by a program

**abstraction** a method used in the **analysis** stage of the program development life cycle; the key elements required for the solution to the problem are kept and any unnecessary details and information that are not required are discarded

**accelerometer** a sensor that measures acceleration and deceleration and that can detect, for example, the orientation of a device

**access levels** different levels of access in a computer system allowing a hierarchy of access levels depending on user's level of security

**access point mapping** see **wardriving**.

**accumulator (ACC)** a temporary general-purpose **register** that stores numerical values at any part of a given operation

**acknowledgement** a message sent to the receiver indicating that data has been received correctly (used in the **ARQ** error detection method)

**active host** functioning software that a **virus** can affect by attaching itself to the code or by altering the code to allow the virus to carry out its attack

**actuator** an output device that converts electrical energy into mechanical movement.

**adaptive cruise control** the use of sensors, actuators and microprocessors to ensure that a vehicle keeps a safe distance behind another vehicle

**ADC** see **analogue to digital converter**

**address** a label for a memory location used by the CPU to track data

**address bus** the **system bus** that carries the **addresses** throughout the computer system

**administrator** a person responsible for the upkeep and maintenance of a computer system that involves multi-user function

**adware** a type of **malware** that attempts to flood the end-user with unwanted advertising

**AI** see **artificial intelligence**

**algorithm** an ordered set of steps to solve a problem

**ALU** see **arithmetic and logic unit**

**analogue to digital converter (ADC)** a device that converts analogue data (for example, data read from sensors) into a form understood by a computer

**analysis** part of the program development life cycle; a process of investigation, leading to the specification of what a program is required to do

**anode** a positive electrode

**anti-spyware** software that detects and removes **spyware** programs installed on a system; the software is based on typical spyware rules or known spyware file structures

**anti-virus software** software that detects, **quarantines** and deletes files or programs infected by a computer **virus**; the software can run in the background or be initiated by the user

**arguments** the actual values that are passed to a **procedure** or **function**

**arithmetic and logic unit (ALU)** the component of the CPU that carries out all arithmetic and logical operations

**arithmetic operator** an **operator** that is used to perform calculations

**array** a data structure containing several elements of the same **data type**; these elements can be accessed using the same identifier name

**ARQ** see **automatic repeat request**

**artificial intelligence (AI)** a collection of rules and data which gives a computer system the ability to reason, learn and adapt to external stimuli

**ASCII code** a character set for all the characters on a standard keyboard, plus special control codes

**assembler** a computer program that translates programming code written in **assembly language** into **machine code**

**assembly language** a programming language that is dependent on computer hardware; a program written in assembly language needs to be translated into **machine code** before it is executed

**asymmetric encryption** a type of **encryption** that uses public keys and private keys to ensure that data is secure

**attribute** something that defines the **objects** stored in a **knowledge base**

**audio compression** a method used to reduce the size of a sound file using perceptual music shaping

**authentication** the process of proving a user's identity by using something they know, something they have or something unique to them

**automated system** a combination of software and hardware designed and programmed to work automatically without the need for any human intervention

**automatic repeat request (ARQ)** a method of checking transmitted data for errors; it makes use of **acknowledgement** and **timeout** to automatically request re-sending of data if the time interval before positive acknowledgement is too long

**autonomous** the ability to operate independently without any human input

**backing store** a secondary storage device (such as an HDD or SSD) used to store data permanently even when the computer is powered down

**back-up** make copies of files onto another storage media in case the original file becomes corrupted or is deleted

**bandwidth** the maximum rate of transfer of data across a network, measured in kilobits per second (kbps) or megabits per second (Mbps)

**barcode** a series of dark and light lines of varying thicknesses used to represent data; the code has to be scanned using a laser or LED light source

**basic input/output system (BIOS)** a suite of programs on **firmware** that are used to perform the initialisation of a computer system during the boot-up process

**bidirectional** can travel in both directions; used to describe **data** that can travel in both directions

**binary number system** a number system based on 2, in which only the digits 0 and 1 are used

**binder 3D printing** a 3D printing method that uses a two-stage pass; the first stage uses dry powder and the second stage uses a binding agent

**biometrics** type of **authentication** that uses a unique human characteristic, such as fingerprints, voice or retina blood vessel pattern

**BIOS** see **basic input/output system**

**bit** the basic computing element that is either 0 or 1, and is formed from the words **bi**nary dig**it**

**bit depth** the number of bits used to represent the smallest unit in a sound file (see also **sampling resolution**)

**bitmap image** an image made up of **pixels**

**blockchain** a decentralised **database** where all transactions are stored; it consists of a number of interconnected computers but not a central server

**Boolean algebra** a form of algebra linked to **logic circuits** and based on TRUE or FALSE

**Boolean operator** an **operator** that is used with **logical operators** to form more complex expressions

**boot up/bootstrap loader** a small program that is used to load other programs to correctly 'start up' a computer system

**boundary data** the largest/smallest data value that is accepted by a program and the corresponding smallest/largest rejected data value

**browser** see **web browser**

**brute force attack** a 'trial and error' method used by cybercriminals to crack passwords by finding all possible combinations of letters, numbers and symbols until the password is found

**bubble sort** an **algorithm** that makes multiple passes through a list comparing each element with the next element and swapping them if necessary. This continues until there is a pass where no more swaps are made

**buffer** a **memory** area used to store data temporarily

**cache** temporary **memory** that uses **static RAM** to hold data/instructions frequently used by the CPU, thereby increasing CPU performance. More generally, cache means any area of storage used to quickly access frequently used data. Other examples include web cache, database cache and DNS cache

**capacitive touch screen** a type of **touch screen** that uses the change in the screen's capacitance (the ability to store an electrical charge) when it is touched by a finger or stylus

**cathode** a negative electrode

**char** a variable or **constant** that is a single character

**character set** a list of characters that have been defined by computer hardware and software; the character set is necessary so that the computer can understand human characters

**charge couple device (CCD)** a light-sensitive cell made up of millions of tiny sensors acting as photodiodes

**chatbot** a pop-up **robot** on a website that appears to enter into a meaningful conversation with a web user

**check digit** an additional digit appended to a number to check if the entered number is error-free; check digit is a data entry check and not a data transmission check

**checksum** a **verification** method used to check if data transferred has been altered or corrupted; calculated from the block of data of data being sent, the checksum value is sent after each data block

**ciphertext** encrypted data that is the result of putting a plaintext message through an **encryption algorithm**

**clock cycle** clock speeds are measured in terms of GHz; this is the vibrational frequency of the **system clock** which sends out pulses along the **control bus**; for example, a 3.5 GHz clock cycle means 3.5 billion clock cycles a second

**cloud storage** a method of data **storage** in which data is stored on offsite servers; the physical storage may be on hundreds of servers in many locations

**coding** part of the program development life cycle; the writing of the program or suite of programs

**cognitive** relating to the mental processes of the human brain involved in acquiring knowledge and understanding through thought, experiences and input from the five senses

**colour depth** the number of **bits** used to represent the colours of a **pixel**

**command line interface (CLI)** an interface which allows communication with the computer by typing in commands using a keyboard

**compiler** a computer program that translates a source program written in a **high-level language** to **machine code**

**complementary metal oxide semi-conductor (CMOS)** a chip that generates an electric current (or pulses) when light falls on its surface

**compression** reduction of the size of a file by removing repeated or redundant pieces of data; this can be **lossy** or **lossless**

**computed tomographic (CT) scanner** technology that can create a 3D image of a solid object by slicing up the object into thin layers (tomography)

**computer aided design (CAD)** software used to create drawings (for example, to send to a 3D printer or to produce blueprints of a microprocessor design)

**constant** a named data store that contains a value that does not change during the execution of a program

**contiguous** next to each other

**control bus** the **system bus** that carries signals from the **control unit** to all other computer components

**control unit** the component of a computer's CPU that ensures synchronisation of data flow and programs throughout the computer by sending out control signals along the **control bus**

**controller** a microprocessor that is in control of a process

**cookie** a text file sent from a website to a user's browser; it is used to remember user preferences each time they visit the website

**core** a unit on a CPU made up of an **ALU, control unit** and **registers**; a CPU may contain a number of cores

**COUNT** an **SQL** command that counts the number of **records** (rows) in which the **field** (column) matches a specified condition; used with SELECT

**counting** keeping track of the number of times an action is performed

**cryptocurrency** a form of **digital currency** that uses a chain of decentralised computers to control and monitor transactions

**cryptography** the protection of data/information by use of coding; it usually involves encryption and decryption

**current instruction register (CIR)** a **register** that stores the current instruction being decoded and executed

**cyclic redundancy check (CRC)** an error checking method in which all the 1-bits in the **data packet payload** are added and the total is stored in the **packet trailer**; the same calculation is repeated at the receiving station

**DAC** see **digital to analogue converter**

**data bus** the **system bus** that allows data to be carried from CPU to memory (and vice versa) or to and from input/output devices

**data interception** unauthorised stealing or observation of data from a wired or wireless network transmission; cybercriminals often use **packet sniffing** or **access point mapping/wardriving** to intercept data

**data packet** a small part of a message/data that is transmitted over a network; after transmission all the data packets are reassembled to form the original message/data

**data redundancy** the unnecessary storing of the same data on several storage devices at the same time

**data type** a classification of how data is stored and displayed, and of which operations can be performed on the stored value

**database** a persistent structured collection of data that allows people to extract information in a way that meets their needs

**debugging** finding errors in a computer program by running or tracing the program

**declare** define the value and **data type** of a **variable** or **constant**

**decomposition** a method used in the **analysis** stage of the program development life cycle; a complex problem is broken down into smaller parts, which can then be sub divided into even smaller parts that can be solved more easily

**defragmentation** a process that reorganises sectors on an HDD by rearranging blocks of data so that they are **contiguous**

**denial of service (DoS) attack** a cyberattack in which cybercriminals seek to disrupt the normal operation of a website by flooding it with requests; also used to clog up a user's mailbox by sending out thousands of **spam** emails; see also **distributed denial of service (DDoS) attack**

**descriptor** a collection of information about a device plugged into a **USB** port; this can be vendor ID (VID), product ID (PID) or serial number

**design** part of the program development life cycle; uses the program specification from the **analysis** stage to show to how the program should be developed

**device driver** software that communicates with the operating system and translates data into a format understood by the device

**digital currency** currency (a system of money) that exists in electronic form only; it has no physical form and is essentially data on a **database**

**digital micromirror device (DMD)** a chip that uses millions of tiny mirrors on its surface to create a video display

**digital signal processor (DSP)** a processor that calculates, for example, the coordinates of a **pointing device** based on the pulses of electricity received

**digital to analogue converter (DAC)** a device that converts digital data into electric currents that can drive motors, actuators and relays, for example

**direct 3D printing** a 3D printing technique in which the print head moves in the x, y and z directions

**disk thrashing** a problem in a hard disk drive (HDD) caused by excessive swapping in and out of data which requires a high rate of head movements during **virtual memory** operations

**distributed control system (DCS)** a powerful computer system programmed to monitor and control a complex process without the need for human interaction

**distributed denial of service (DDoS) attack** a **denial of service (DoS) attack** in which the fake requests come from many different computers, which makes it harder to stop

**DIV** an **arithmetic operator** that returns the quotient (whole number part) of a division; different languages use different symbols for this operation

**DNS cache poisoning** altering **IP addresses** on a **domain name server (DNS)** with the intention of redirecting a user's browser to a fake website; carried out by a pharmer (see **pharming**) or hacker (see **hacking**)

**domain name server (DNS)** a **server** that looks up domain names for websites (e.g. **www.hoddereducation.com**) in order to find the **IP addresses** that a computer needs to locate the web servers (e.g. 107.162.140.19)

**drone** a flying **robot** that can be **autonomous** or operated using remote control; a drone can be used for reconnaissance or deliveries

**dual core** a CPU containing two **cores**

**dual layering** using two recording layers in storage media such as DVDs and some Blu-rays

**dynamic host configuration protocol (DHCP)** a server that automatically provides and assigns an **IP address**

**dynamic IP address** a temporary **IP address** assigned to a device each time it logs onto the **network**

**dynamic RAM (DRAM)** a type of **RAM** chip that needs to be constantly **refreshed**

**eavesdropper** another name for a hacker who intercepts data being transmitted on a wired or wireless network

**echo check** a method used to check if data has been transferred correctly; data is sent to a receiver and then immediately sent back to the sender; the sender then checks if the received data matches the sent data

**EEPROM** stands for electronically erasable programmable **ROM**

**embedded system** a combination of hardware and software designed to carry out a specific set of functions

**encryption** the process of making data meaningless using encryption keys; without the correct decryption key the data cannot be decoded

**encryption algorithm** a complex piece of software that takes **plaintext** and generates an encrypted **string** known as **ciphertext**

**end-effector** an attachment to a **robot** arm that allows it to carry out a specific task, such as spray painting

**error code** an error message generated by the computer

**error handling routine** a routine in a program or **operating system** that recognises and recovers a system from abnormal **inputs** or hardware faults (for example, recovery from an attempt to divide by zero)

**expert system** a form of **AI** that has been developed to mimic a human's knowledge and expertise

**explanation system** part of an **expert system** which informs the user of the reasoning behind its conclusions and recommendations

**extreme data** the largest/smallest data value that is accepted by a program

**Fetch–Decode–Execute cycle** a cycle in which instructions and data are fetched from memory, decoded and finally executed

**field** a single item of data in a record, stored in a database table

**file** a collection of data stored by a computer program to be used again

**file management** part of an **operating system** that manages files in a computer (for example, the ability to create, delete, copy, open, close and rename files)

**firewall** software or hardware that sits between a computer and an external network (for example, the **internet**); the firewall monitors and filters all incoming and outgoing traffic

**firmware** a program that provides low-level control for devices

**flip flop** an electronic circuit with only two stable conditions

**flowchart** a diagram that shows the steps required for a task (sub-system) and the order in which the steps are to be performed.

**format check** a check that the characters entered conform to a pre-defined pattern

**frame QR code** a type of **QR code** that includes a space for advertising

**FROM** an **SQL** command that identifies the **table** to use

**full-duplex** data that can be sent in both directions at the same time (simultaneously)

**function** a set of programming statements grouped together under a single name which can be called to perform a task in a program, rather than including a copy of the code every time; just like a **procedure** except a function will return a value back to the main program

**graphical user interface (GUI)** an interface that uses icons to represent apps and tasks which the user can select/launch by clicking on a mouse or using a **touch screen**

**hacking** the act of gaining illegal access to a computer system without the owner's permission

**half-duplex** data that can be sent in both directions but not at the same time

**hardware management** the part of an **operating system** that controls all input and output devices; it is made up of sub-systems such as printer management

**heuristic checking** checking software for behaviour that could indicate a possible **virus**

**hexadecimal number system** a number system based on the value 16, which uses the denary digits 0 to 9 and letters A to F

**high-level language (HLL)** a programming language that is independent of computer hardware; a program written in a HLL needs to be translated into **machine code** before it is executed

**hopping/hop number** a number in a **data packet** header used to stop **data packets** that never reach their destination from 'clogging up' the data paths/routes

**HTML** see **hypertext mark-up language**

**human computer interface (HCI)** an interface supplied by the **operating system** to 'hide' the complexities of the software and hardware from the human user

**hyperlink** highlighted text or an image that is activated by clicking and which links to further text, images, a web page or a website

**hypertext mark-up language (HTML)** the language used to design, display and format web pages, and to write http(s) protocols

**hypertext transfer protocol (http)** a set of rules that must be obeyed when transferring web pages over the **internet**

**hypertext transfer protocol secure (https)** **http** with extra security (such as **SSL**) applied

**IDE** see **integrated development environment**

**image resolution** the number of **pixels** in the X–Y direction of an image, for example, 4096 × 3192 pixels

**immediate access store (IAS)** memory that holds all data and programs needed to be accessed by the **control unit**

**index** identifies the position of an element in an **array**

**inference engine** a kind of search engine used in an **expert system** which examines the **knowledge base** for information that matches the queries

**inference rules** rules used by the **inference engine** in an **expert system** to draw conclusions using IF statements

**infrared touch screen** a type of **touch screen** that uses infrared beams and sensors to detect where the screen has been touched

**inputs** the data used by the system that needs to be entered while the system is active

**instruction set** the complete set of **machine code** instructions used by a particular microprocessor

**integer** a positive or negative whole number that can be used with mathematical **operators**

**integrated circuit** usually a chip made from a semi-conductor material which carries out the same tasks as a larger circuit made from individual components

**integrated development environment (IDE)** a suite of programs used to write and test a computer program written in a **high-level language**

**internet** the world-wide interconnection of networks; the internet makes use of TCP and IP protocols

**interpreter** a computer program that analyses and executes a program written in a **high-level language** line by line

**interrupt** a signal sent from a device or software to a microprocessor requesting its attention; the microprocessor suspends all operations until the interrupt has been **serviced**

**interrupt priority** **interrupts** are given priorities so that the microprocessor knows which one needs to be **serviced** first and which interrupts are to be dealt with quickly

**interrupt service routine (ISR)** software that handles **interrupt** requests (for example, when the printer is out of paper) and sends the request to the CPU for processing

**IP address** short for internet protocol address and identified as either IPv4 or IPv6; the IP address gives a unique **address** to a network, identifying its location

**iteration** a section of programming code that can be repeated under certain conditions

**jpeg** from Joint Photographic Expert Group; a form of **lossy file compression** used with image files which relies on the inability of the human eye to distinguish certain colour changes and hues

**key field** the **field** that uniquely identifies a **record** in a file

**knowledge base** a repository of facts which is a collection of **objects** and **attributes**

**latency** the lag in a system; for example, the time it takes to find a track on a hard disk, which depends on the time it takes for the disk to rotate around to its read-write head

**length check** a method used to check that the data entered is a specific number of characters long or that the number of characters is between an upper value and a lower value

**library routine** functions or procedures that are included with a programming language

**LiDaR** a contraction of light detection and ranging; the use of lasers to build up a 3D image of the surroundings

**linear search** an **algorithm** that inspects each item in a list in turn to see if the item matches the value being searched for

**logic circuit** this is formed from a combination of **logic gates** and designed to carry out a particular task; the **output** from a logic circuit will be 0 or 1

**logic gate** an electronic circuit that relies on 'on/off' logic; the most common gates are NOT, AND, OR, NAND, NOR and XOR

**logical operator** an **operator** that is used to decide the path to take through a program if the expression formed is true or false

**logical shift** an operation that shifts **bits** to the left or right in a **register**; any bits shifted out of a register (left or right) are replaced with zeroes

**lossless (file compression)** a file **compression** method that allows the original file to be fully restored during the decompression process, e.g. **run length encoding (RLE)**

**lossy (file compression)** a file **compression** method in which parts of the original file cannot be recovered during the decompression process, e.g. JPEG, **mp3**

**loudspeaker** an output device that converts electric current into sound

**low-level language (LLL)** a programming language that is dependent on computer hardware; both **machine code** and **assembly language** are LLLs

**MAC address** standing for Media Access Control, this address uniquely identifies a device on the **internet**; it takes the form: NN-NN-NN-DD-DD-DD, where NN-NN-NN is the manufacturer code and DD-DD-DD is the device code

**machine code** a **binary** programming language; a program written in machine code can be loaded and executed without translation

**machine learning** a sub-set of **AI** in which **algorithms** are trained and learn from past experiences and examples

**malware** programs (such as viruses, worms and Trojan horses) that are installed on a user's computer with the aim of deleting, corrupting or manipulating data illegally

**memory** the devices within the computer that are directly accessible by the CPU; there are two types of memory – **RAM** and **ROM**; memory is different to hard disk drives, for example, which are known as storage devices

**memory address register (MAR)** a **register** that stores the **address** of the **memory location** currently being read from or written to

**memory data register (MDR)** a **register** that stores data that has just been read from **memory** or data that is about to be written to memory

**memory location** a numbered place in **memory** where values can be stored

**memory management** the part of an **operating system** that controls main **memory**

**MOD** an **arithmetic operator** that returns the remainder of a division; different languages use different symbols for this operation

**MP3** a **lossy file compression** method used for music files

**MP4** a **lossy file compression** method used for multimedia files

**multitasking** a function that allows a computer to process more than one task/process at a time

**nesting** the inclusion of one type of code construct inside another

**network interface card (NIC)** a hardware component (circuit board or chip) that is required to allow a device to connect to a network, such as the **internet**

**node** stages in a network that can receive and transmit **data packets**; **routers** are nodes in communication networks

**normal data** data that is accepted by a program

**object** in an expert system, an item stored in a **knowledge base**

**optical character recognition (OCR)** technology that can convert hard copy text or images into a digital format to be stored in a computer memory

**organic LED (OLED)** a light-emitting diode that uses the movement of electrons between a **cathode** and an **anode** to produce an on-screen image; it generates its own light so no backlighting is required

**opcode** part of a **machine code** instruction that identifies what action the CPU has to perform

**operand** part of a **machine code** instruction that identifies what data is to be used

**operating system** software that provides both an environment in which applications can run and an interface between computer and human operator

**operator** a special character or word in a programming language that identifies an action to be performed

**optical mouse** a **pointing device** that uses a red LED to track the movement of the device and then relays its coordinates back to a computer

**optical storage** a type of storage that uses laser light to read and write data, and includes CDs, DVDs and Blu-ray discs

**ORDER BY** an SQL command that sorts the results from a query by a given column, either alphabetically or numerically

**output** information that needs to be displayed or printed for the users of the system

**overclocking** changing the clock speed of a **system clock** to a value higher than the factory/recommended setting

**overflow error** the result of carrying out a calculation that produces a value that is too large for the computer's allocated word size (8-bit, 16-bit, 32-bit, etc.)

**packet header** the part of the **data packet** that contains the **IP addresses** of the sender and receiver, and includes the packet number which allows reassembly of the data packets

**packet sniffing** a method used by a cybercriminal to examine **data packets** being sent over a network and to find the contents of a data packet, which are sent back to the cybercriminal

**packet switching** a method of transmission in which a message is split into many **data packets** which can then be sent along pathways independently of each other

**packet trailer** the part of a **data packet** that indicates the end of the data packet and **cyclic redundancy check** error check

**parallel data transmission** sending data down several channels/wires several **bits** at a time (usually 1 byte)

**parameters** the **variables** in a **procedure** or **function** declaration that store the values of the arguments passed from the main program to the procedure or function

**parity bit** a **bit** (either 0 or 1) added to a byte of data in the most significant bit position; this ensures that the byte follows the correct even parity or odd parity protocol

**parity block** a horizontal and vertical **parity check** on a block of data being transmitted

**parity byte** an extra byte of data sent at the end of a **parity block**; it is composed of the **parity bits** generated from a vertical **parity check** of the data block

**parity check** a method used to check whether data has been transferred correctly; it makes use of even parity (an even number of 1-bits) or odd parity (an odd number of 1-bits)

**patch** an update for software that is developed to improve the software and/or to remove any bugs

**payload** the actual data being carried in a **data packet**

**persistent cookies** a **cookie** that is stored on the user's hard drive and is only deleted when the expiry date is reached or the cookie is deleted by the user

**pharming** redirecting a user to a fake website in order to illegally obtain personal data about the user without their knowledge; unlike **phishing**, pharming is initiated without needing any action by the user

**phenotyping** the process of observing the physical characteristics of a plant to assess its health and growth

**phishing** sending out legitimate-looking emails designed to trick the recipients into giving their personal details to the sender of the email

**piezoelectric crystal** a crystal located in an ink reservoir within an inkjet printer; the crystal vibrates and forces ink out onto paper

**pixel** the smallest element used to make up an image; a contraction of the term picture element; each pixel is represented by a minimum of 8 **bits** and 3 pixels are needed to make up the RGB colours

**pixel density** derived from the term 'picture element', the number of **pixels** per square inch in a digital image

**pixelated (image)** this is the result of zooming into a **bitmap image**; on zooming out, the **pixel** density can be diminished to such a degree that the actual pixels themselves can be seen

**plaintext** the original text/message before it is put through an **encryption algorithm**

**pointing device** an input device that allows the user to control the movement of an on-screen cursor or to select items on-screen by clicking a button on the device

**post-WIMP** a modern **touch screen** interface system that allows actions such as pinching and rotating

**presence check** a check that a data item has been entered

**prettyprinting** displaying source code using different colours and formatting, which make the code easier to read and understand

**primary key** a **field** in a **database** that uniquely identifies a **record**

**privacy settings** controls available on social networking sites and other websites which allow users to limit who can access their profile or what they are allowed to see

**private key** a type of **encryption** key that is known only to a single computer/user

**procedure** a set of programming statements grouped together under a single name which can be called to perform a task in a program, rather than including a copy of the code every time the task is performed

**processes** the tasks that need to be performed by a program using the input data and any other previously stored data

**program counter (PC)** a **register** that stores the **address** of where the next instruction to be read can be found

**proof-of-work** the **algorithm** used in **blockchain** networks to confirm a transaction and to produce new blocks to add to the chain; special users called miners complete and monitor transactions on the network for a reward

**proxy server** a server that acts as an intermediary server through which **internet** requests are processed; it often makes use of **cache** memory to speed up web page access

**pseudocode** a simple method of showing an **algorithm**; it describes what the algorithm does using English key words that are very similar to those used in a high-level programming language but without the strict syntax rules

**public key** a type of **encryption** key that is known to all users

**QR code** see **quick response code**

**quad core** a CPU containing four **cores**

**quantum computer** a computer that can perform very fast calculations; it can perform calculations that are based on probability rather than simple 0 or 1 values; this gives a quantum computer the potential to process considerably more data than existing computers

**quarantine** to isolate (in order to later delete) a file or program identified by **anti-virus software** as being infected by a **virus**

**quick response (QR) code** a matrix of dark and light squares which represent data; the pattern can be read and interpreted using a smartphone camera and QR app

**RAM** see **random access memory**

**RANDOM** a library routine that generates a random number

**random access memory (RAM)** primary **memory** that can be written to or read from

**range check** a check that the value of a number is between an upper value and a lower value

**ransomware** a type of **malware** that encrypts data on a user's computer and 'holds the data hostage' until a ransom is paid

**read only memory (ROM)** primary memory that cannot be written to (changed) and can only be read

**real number** a positive or negative number with a fractional part; real numbers can be used with mathematical **operators**

**real time streaming** the transmission of data over a network for live events where the data is sent as soon as it is received or generated

**record** a collection of **fields** that describe one item

**refresh** the need to recharge every few seconds in order to maintain charge; for example, with a device such as a capacitor

**register** a temporary component in the CPU which can be general or specific in its use; it holds data or instructions as part of the **Fetch–Decode–Execute** cycle

**repetitive strain injury (RSI)** pain felt in the muscles, nerves and tendons caused by a repetitive action (for example, excessive clicking of a mouse button over a period of time)

**report window** a separate window in the runtime environment of an **IDE** that shows the contents of **variables** during the execution of a program

**resistive touch screen** a type of **touch screen** that uses two conductive layers which make contact where the screen has been touched

**RLE** see **run length encoding**

**robot** a mechanical device that can carry out tasks normally done by humans

**robotics** the branch of (computer) science that encompasses the design, construction and operation of robots

**ROM** see **read only memory**

**ROUND** a library routine that rounds a value to a given number of decimal places

**router** a device that enables **data packets** to be moved between different networks, for example, to join a LAN to a WAN

**rules base** a collection of **inference rules** used to draw conclusions

**run length encoding (RLE)** a **lossless file compression** method used to reduce the size of text and photo files in particular

**sampling rate** the number of sound samples taken per second in digital sound recording

**sampling resolution** the number of **bits** used to represent sound amplitude in digital sound recording (also known as **bit depth**)

**screensaver** software that supplies a still or moving image on a monitor if a computer has been inactive for a period of time

**search bot** see **WebCrawler**

**secure sockets layer (SSL)** a security protocol used when sending data over a network (such as the **internet**)

**security management** the part of an **operating system** that ensures the integrity, confidentiality and availability of data

**SELECT** a **SQL** command that fetches specified **fields** (columns) from a **table**

**selection** allowing the selection of different paths through the steps of a program

**sequence** the order in which the steps in a program are executed

**serial data transmission** sending data down one channel/wire one **bit** at a time

**service (an interrupt)** when an interrupt is received, some action needs to be taken by the processor depending on what caused the interrupt; until this is resolved (that is, it is serviced), the interrupt cannot be removed to allow the processor to continue

**session cookie** a **cookie** that is stored temporarily on a computer; it is deleted when the browser is closed or the website session ends

**set of test data** all the items of data required to work through a solution

**simplex** data that can be sent in one direction only

**single-table database** a **database** that contains only one **table**

**skewed data** data that arrives at the destination with the **bits** no longer synchronised

**social engineering** manipulating people into breaking normal security procedures (such as giving away their passwords) in order to gain illegal access to computer systems or to place **malware** on their computer

**spam** unsolicited emails sent to a user's mailbox

**spear phishing** similar to **phishing** but targeting specific people or organisations rather than carrying out a blanket attack

**spyware** a type of **malware** that gathers information by monitoring a user's activities on a computer and sending the gathered information back to the cybercriminal who sent out the spyware

**SQL** see **Structured Query Language**

**SQL scripts** a list of **SQL** commands that perform a given task, often stored in a file so the script can be reused

**SSD endurance** the total guaranteed number of times that data can be written to or read from a solid state drive (SSD) in its usable life cycle

**SSL** see **secure sockets layer**

**SSL certificate** a form of digital certificate which is used to authenticate a website; if the SSL certificate can be authenticated, any communication or data exchange between browser and website is secure

**static IP address** an **IP address** that doesn't change

**static RAM (SRAM)** a type of **RAM** chip that uses **flip flops** and doesn't need to be constantly **refreshed**

**storage** data that needs to be stored in files on an appropriate media for use in the future

**string** a **variable** or **constant** that is several characters in length; strings vary in length and may even have no characters (an empty string); the characters can be letters and/or digits and/or any other printable symbol

**structure diagram** a diagram that shows the design of a computer system in a hierarchical way, with each level giving a more detailed breakdown of the system into sub-systems

**Structured Query Language (SQL)** the standard query language for writing scripts to obtain useful information from a relational **database**

**SUM** an **SQL** command that returns the sum of all the values in a **field** (column); used with SELECT

**symmetric encryption** a type of **encryption** in which the same encryption key is used both to encrypt and to decrypt a message

**system bus** a connection between major components in a computer that can carry data, **addresses** or control signals

**system clock** produces timing signals on the **control bus** to ensure that synchronisation takes place

**table** a collection of related **records** in a **database**

**testing** part of the program development life cycle; systematic checks done on a program to make sure that it works under all conditions.

**thermal bubble** inkjet printer technology whereby tiny resistors create heat and form an ink bubble which is ejected onto paper in an inkjet printer

**thrash point** the point at which the execution of a program comes to a halt because the system is so busy moving data in and out of memory rather than actually executing the program

**timeout** the time interval that is allowed to elapse before an **acknowledgement** is received (in the **ARQ** error detection method)

**timestamp** a digital record of the data and time that a data block is created in **blockchain** networks

**top-down design** the breaking down of a computer system into a set of sub-systems, then breaking down each sub-system into a set of smaller sub-systems, until each sub-system just performs a single action.

**totalling** keeping a total that values are added to

**touch screen** a screen that allows the user to select or manipulate a screen image using the touch of a finger or stylus; touch screens most frequently use capacitive, infra-red or resistive technology

**translator** converts a program written in a high-level language program into machine code

**Trojan horse** a type of **malware** that is designed to look like legitimate software but contains malicious code that can cause damage to a computer system

**truth table** a method of checking the output from a **logic circuit**; a truth table lists all the possible **binary** input combinations and their associated **outputs**; the number of outputs will depend on the number of **inputs**; for example, two inputs have $2^2$ (4) possible binary combinations, three inputs have $2^3$ (8) possible binary combinations, and so on

**two's complement** a method of representing negative numbers in binary; when applied to an 8-bit system, the left-most bit (most significant bit) is given the value –128

**two-step verification** a type of **authentication** that requires two methods of **verification** to prove the identity of a user

**type check** a check that the data entered is of a specific type

**typo squatting** the use by cybercriminals of subtle spelling errors in website addresses in order to trick users into visiting their fake websites

**Unicode** a character set which represents all the languages of the world (the first 128 characters are the same as **ASCII code**)

**unidirectional** travel in one direction only; used to describe data

**uniform resource locator (URL)** a text-based address for a web page

**universal serial bus (USB)** a type of **serial data transmission** which has become the industry standard for connecting computers to devices via a USB port

**USB** see **universal serial bus**

**user account** an agreement that allows an individual to use a computer; the user needs a user name and password to enter the user's area

**user preferences** settings or options stored in **cookies** that can remember customised web pages or browsing history to help target adverts

**utility programs (utilities)** part of an operating system which carries out certain functions such as **virus** checking, **defragmentation** and **screensaver**

**validation** automated checks carried out by a program that data is reasonable before it is accepted into a computer system

**variable** a named data store than contains a value that may change during the execution of a program

**verification** checking that data has been accurately copied from another source and input into a computer or transferred from one part of a computer system to another

**virtual keyboard** an on-screen keyboard which uses the features of the **touch screen** to emulate a physical keyboard

**virtual memory** a **memory management** system that makes use of secondary storage and software to enable a computer to compensate for the shortage of physical **RAM** memory

**virtual shopping basket** an area of memory in a website where items a user wishes to purchase are temporarily stored; items remain in the basket until payment is made or the session has ended

**virus** a program or program code that replicates itself and has been created with the intention of deleting or corrupting files or of causing the computer system to malfunction

**volatile** describes **memory** that loses its contents when the power is turned off

**von Neumann architecture** a type of computer architecture which introduced the concept of the stored program in the 1940s

**wardriving** using a laptop, antenna, GPS device and software to intercept Wi-Fi signals and illegally obtain data; sometimes called **access point mapping**

**web browser** software that connects to a **domain name server (DNS)** to locate **IP addresses**; a browser interprets **HTML** web pages sent to a user's computer so that the user can read documents and watch multimedia

**WebCrawler/search bot** a software **robot** that roams the **internet** scanning websites and categorising them; often used by search engines

**web scraping** a method of obtaining data from websites

**WHERE** a **SQL** command to include only those **records** (rows) in a query that match a given condition

**windows icons menu and pointing device (WIMP)** an interface that uses a **pointing device** such as a mouse to select options from screen icons or a menu

**wired equivalency privacy (WEP) encryption protocol security** an **algorithm** for wireless networks to protect them against **data interception**

**word** a group of **bits** used by a computer to represent a single unit; for example, modern computers often use 64-bit word lengths

**word list** a text file containing a collection of words used in a **brute force attack**

**World Wide Web** a massive collection of web pages that is based on **hypertext transfer protocols** (http and https)

**worm** a stand-alone type of **malware** that can self-replicate; unlike **viruses**, worms don't need an active host – they can spread throughout a network without the need for any action by an end-user

**Created with teachers and students in schools across the globe, Boost is the next generation in digital learning for schools and colleges, bringing quality content and new technology together in one interactive website.**

This Teacher's Guide includes a print handbook and a subscription to Boost, where you will find a range of online resources to support your teaching.

- **Confidently deliver the revised syllabus:** expert author guidance on how to approach the syllabus, practical support to help you deliver topics as well as two schemes of work.
- **Develop key concepts and skills:** let students see how their skills are developing with sample programming files and detailed answers to activities and questions in the accompanying Student's Book, Computer Systems Workbook and Algorithms, Programming and Logic Workbook.
- **Enhance learning:** A glossary and flashcards to support vocabulary plus PowerPoints to explain technical topics.

This resource is endorsed by Cambridge Assessment International Education

- ✓ Provides teacher support for the Cambridge IGCSE, IGCSE (9-1) and O Level Computer Science syllabuses (0478/0984/2210) for examination from 2023
- ✓ Has passed Cambridge International's rigorous quality-assurance process
- ✓ Developed by subject experts
- ✓ For Cambridge schools worldwide

For over 25 years we have been trusted by Cambridge schools around the world to provide quality support for teaching and learning. For this reason we have been selected by Cambridge Assessment International Education as an official publisher of endorsed material for their syllabuses.

Working for over 25 YEARS WITH Cambridge Assessment International Education

Boost
This series includes eBooks and teacher support.
Visit www.hoddereducation.com/boost to find out more.

HODDER EDUCATION
e: education@bookpoint.co.uk
w: hoddereducation.com

ISBN 978-1-398-31850-2
9 781398 318502

FSC